

CS/CE 224/272 - Object Oriented Programming and Design Methodology

Nadia Nasir



Issues with Arrays (1)

- Arrays
 - Collection of a number of values of a similar data type
 - Requires contiguous memory locations
 - Access is $O(1)$, any value can be accessed directly through an indexer (array[i])
 - Searching is $O(N)$ for an array of N elements
- Insertion and Deletion of a value at a location i are cumbersome
 - Insertion: You have to create a new array of size $(N+1)$, copy values by movement of all items at location $>i$ by one place to the right
 - Deletion requires movement of all items at location $>i$ by one place to the left
- Large array allocations are impossible if memory is already scarce

Issues with Arrays (2)

- Insertion of value 10 at index 2
 - Move all values from index 2 to 5 one place to the right

Value	1	4	5	6	3	2
index	0	1	2	3	4	5

1	4	5	6	3	2	
0	1	2	3	4	5	6



1	4	10	5	6	3	2
0	1	2	3	4	5	6

- Deletion of value 4 from index 1
 - Move all values from index 2 to 5 one place to the left

1	4	5	6	3	2
0	1	2	3	4	5



1		5	6	3	2
0	1	2	3	4	5



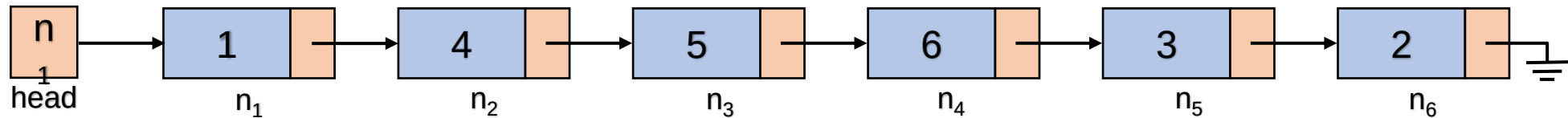
1	5	6	3	2
0	1	2	3	4

Introduction to LinkedList

- Instead of creating a contiguous allocation, we create nodes dynamically on demand and link them through pointers
- No limit on the number of nodes creatable (memory is the only limit)
- Insertion and Deletion are easy, you just need to update a few pointers
- Traversal is $O(N)$
- Reverse Traversal is $O(N^2)$ for Singly LinkedList and $O(N)$ for Doubly LinkedList

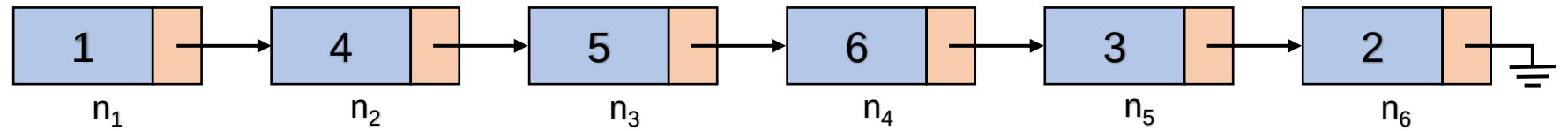
Basic Structure of LinkedList

- The same data that was stored in the array may be stored in a linked list as follows

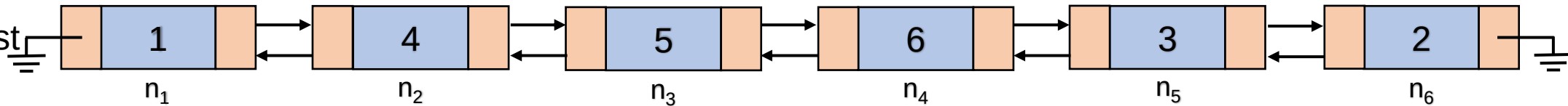


Types of LinkedList

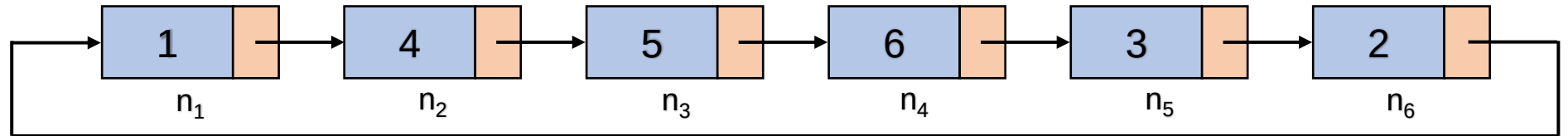
Singly LinkedList



Doubly LinkedList



Circular LinkedList



What is a **struct**?

What is a **struct**?

- Back in C, there were no classes (C is not an OO language).
- However, you could **group data together** in C using **structs**.
 - But they could not contain functions.
- So **structs** used to be just a collection of data.
 - Data types of each data could be different though.

What is a **struct**?

- In C++, we still have **structs**, however, they are very similar to **classes** with a few exceptions.
 - Like classes, you can have **both data and functions** in **structs** too in C++.
 - By default, all members in a **struct** are public.

Author's note

To be pedantic, there's one more minor difference -- structs inherit from other class types publicly and classes inherit privately. We'll cover what this means in the chapter on inheritance, but this particular point is practically irrelevant since you shouldn't rely on the defaults for inheritance anyway.

When to use **structs**?

- By convention, whenever you want to make a **data class** (a class that just contains data members), consider making a **struct**.
- You also should not mind the data members being publicly available.

Linked Lists

Some motivation first...

Whenever you create an array, you have to **specify the size of the array upfront**, whether it's a fixed array or a dynamic array.

```
double arr[20];                                // specify a constant size
```

```
double* arrPtr{ new double[user] };          // specify a user's size
```

Linked lists, however, **don't need an initial size**.

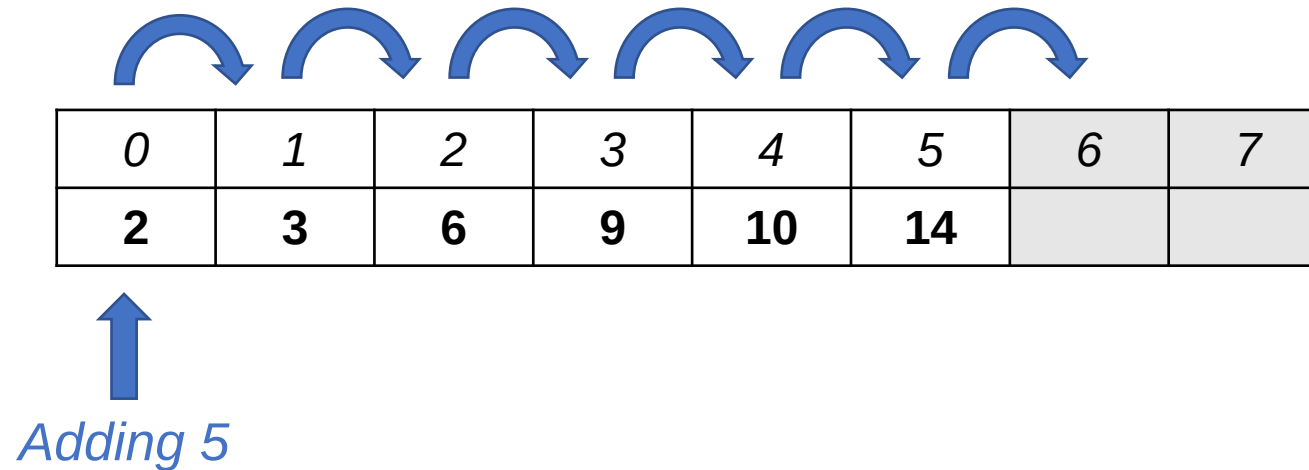
Another thing with linked lists is that once created, they can **grow and shrink** as the programmer desires. However, with arrays, although resizing is possible with dynamically-allocated arrays, the process is more involved.

- Allocate a larger array.
- Copy over the elements from the old array to the new one.
- Deallocate the old array.

Other benefits include **improvements in time complexity** for certain functions compared to arrays.

Let's say you want to insert an element **at the start** of an array, all elements have to be shifted - $O(n)$, n being the number of elements in the array.

However, with linked lists, this **insertion at start** can happen in $O(1)$ (*we'll see how later*).



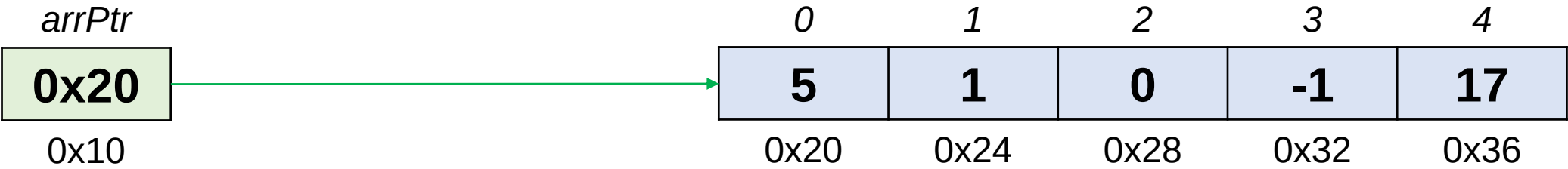
However, certain other functions **degrade in time complexity** compared to an array. For example, accessing an element in an array is $O(1)$ compared to $O(n)$ in a linked list (n being the number of nodes in a linked list). So it's **not a complete replacement for arrays**.

You need to pick the right tool (data structure) for the right job!

Developing LL Structure

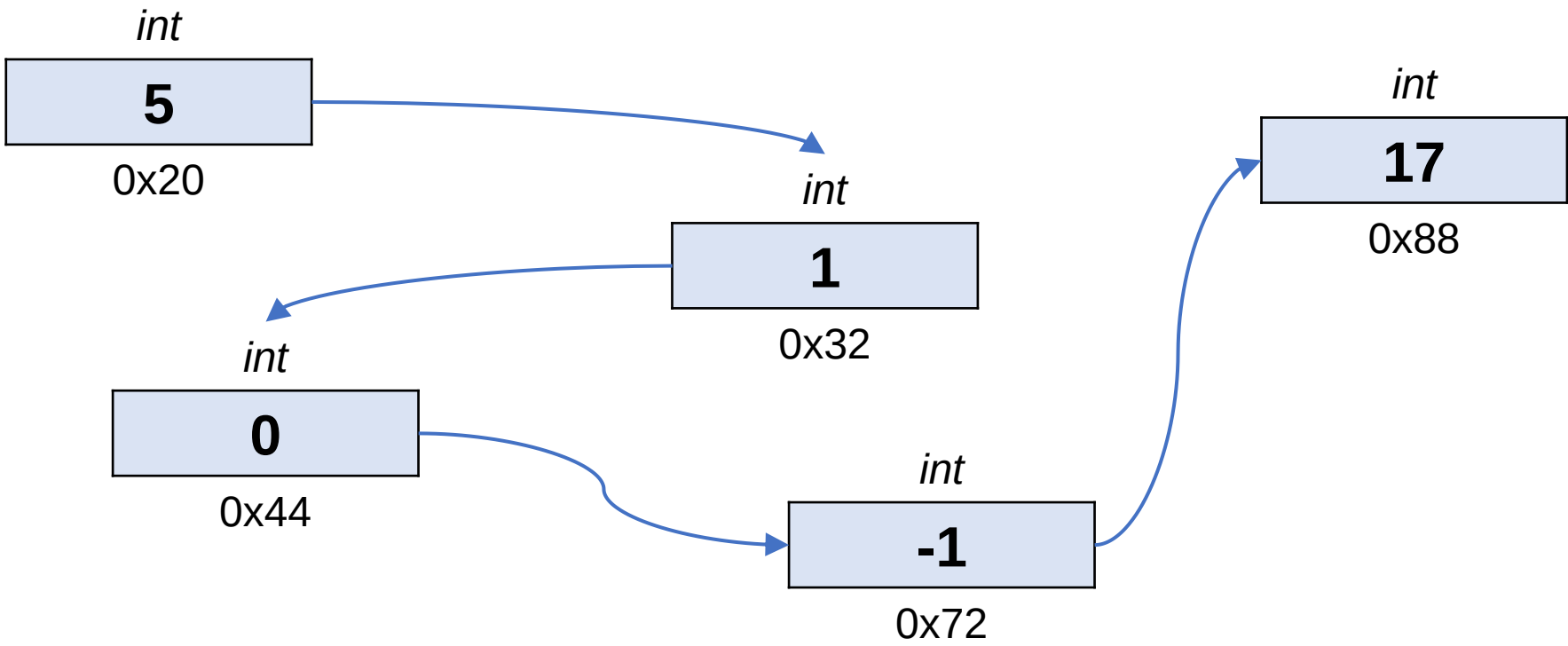
Let's design it.

```
int* arrPtr{ new int[5] {5, 1, 0, -1, 17} };
```

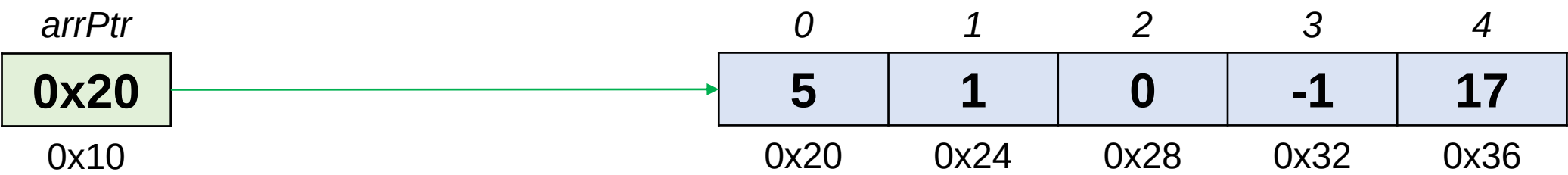


// Some Program to create a LL

We need pointers to create these connections.



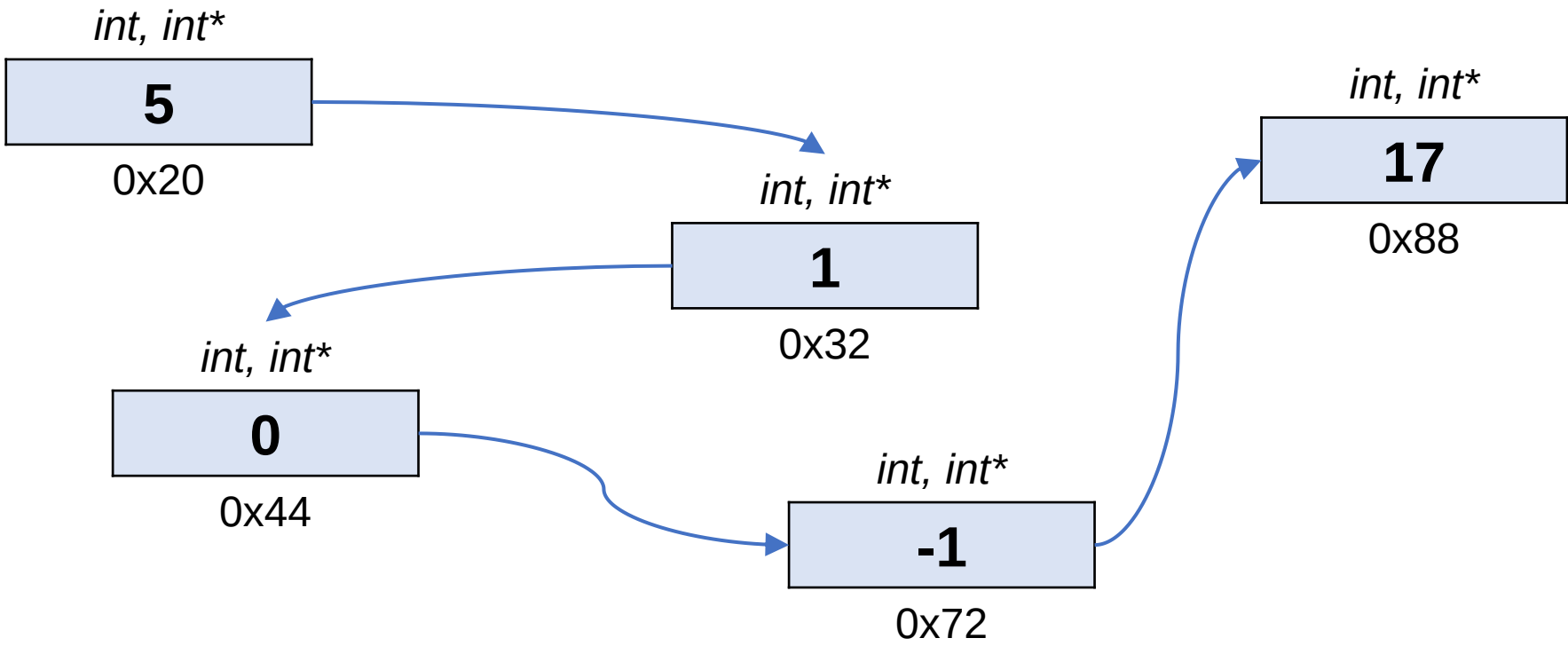
```
int* arrPtr{ new int[5] {5, 1, 0, -1, 17} };
```



// Some Program to create a LL

Okay so I need to add pointers. But what I have is an integer.

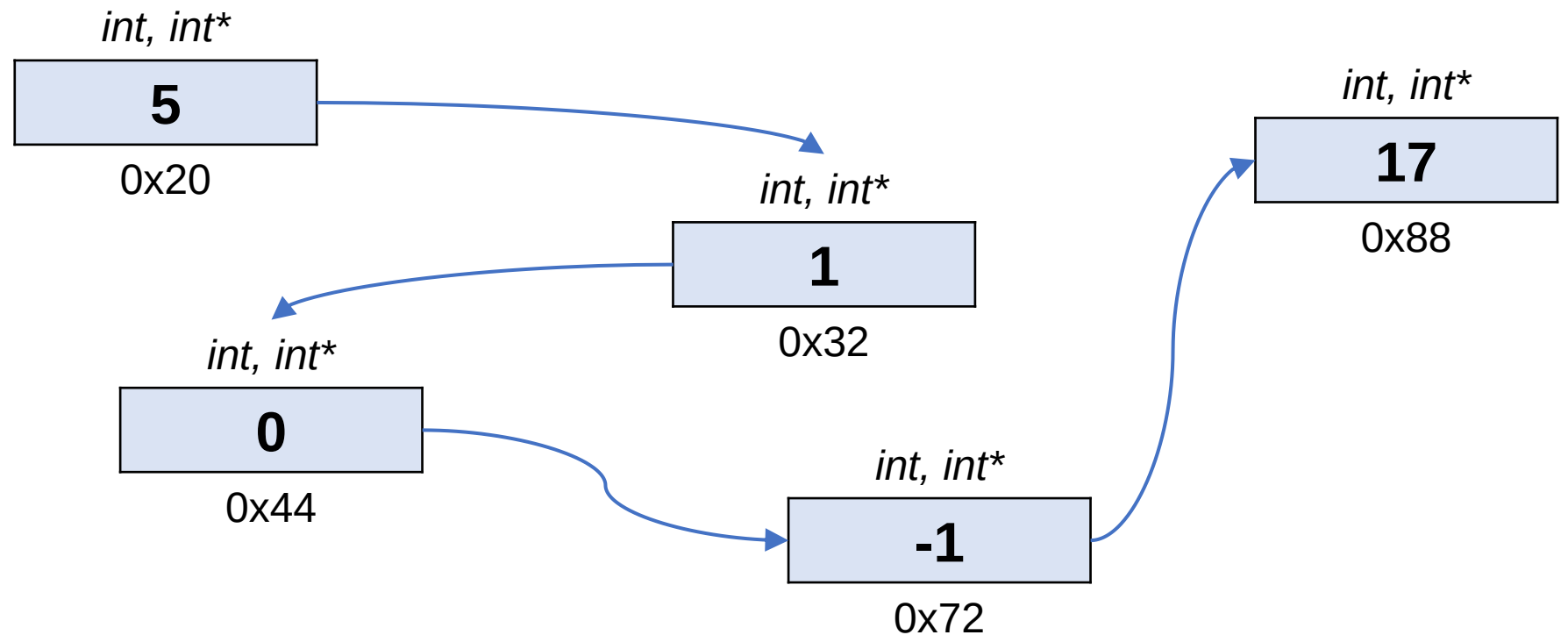
How can I accommodate an int and an int*?



*I can create objects that would contain the `int` and the `int*`.
We conventionally call these objects **Nodes**.*

With this information, can you identify a mistake that I've made?

```
class Node
{
    int m_data;
    int* m_next;
};
```

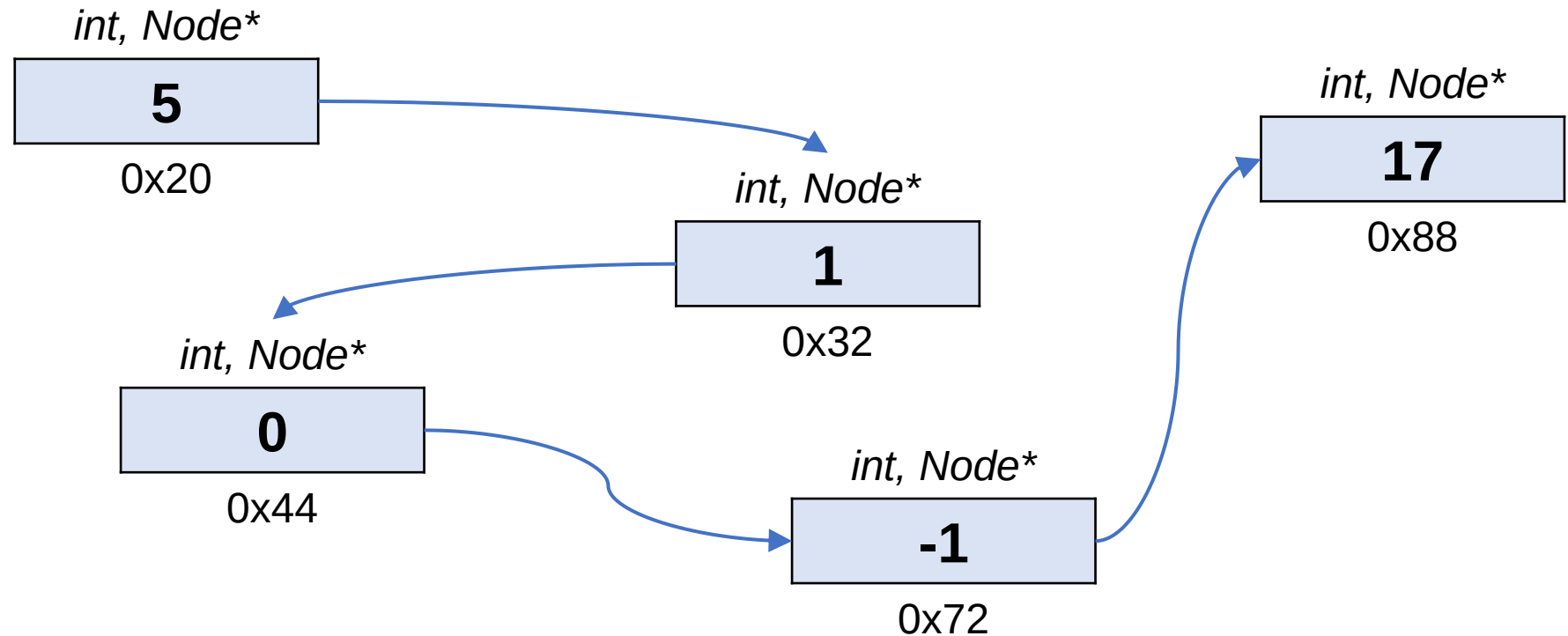


Note that all of these blue rectangles are Node type objects.

How can an **int*** point at an entity whose data type is **Node**?

Our pointer attribute should be the **same type** as the data type of the entity to which it is pointing at – **Node*** in this case.

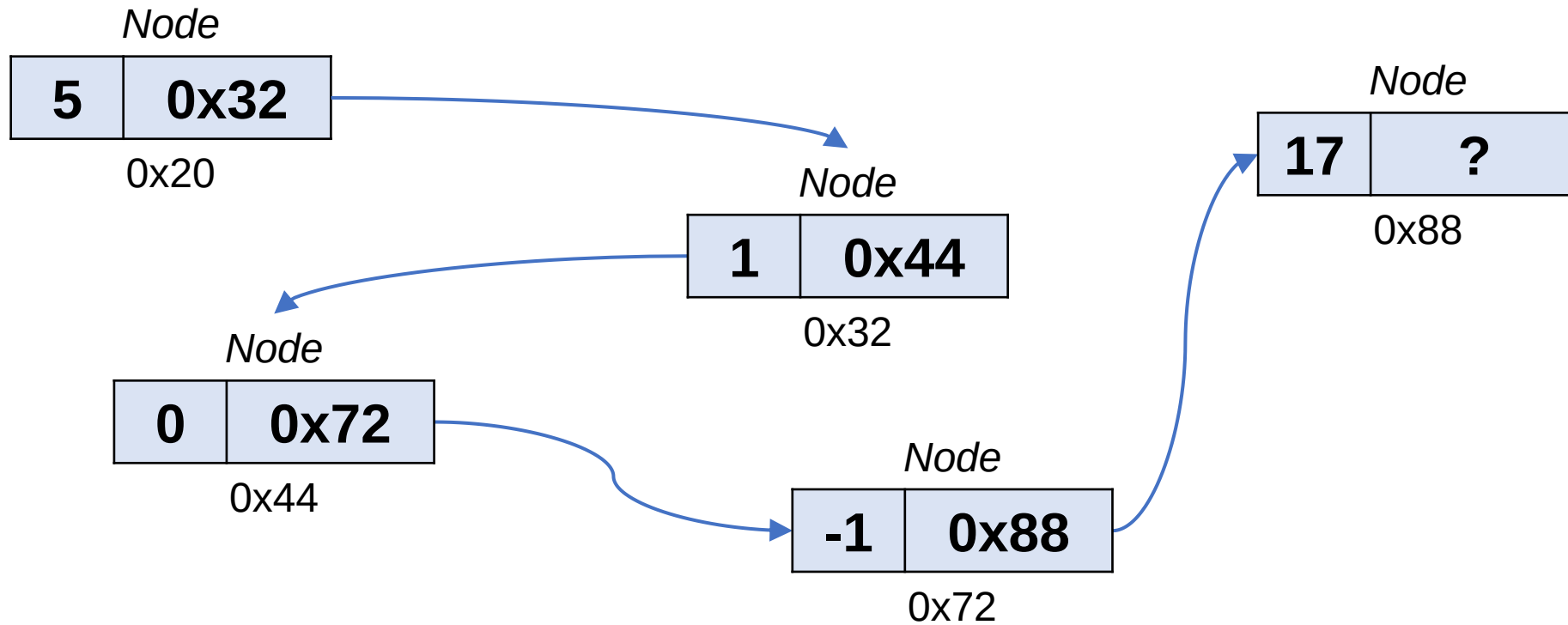
```
class Node
{
    int m_data;
    Node* m_next;
};
```



Updated visual based on all of the discussion thus far.

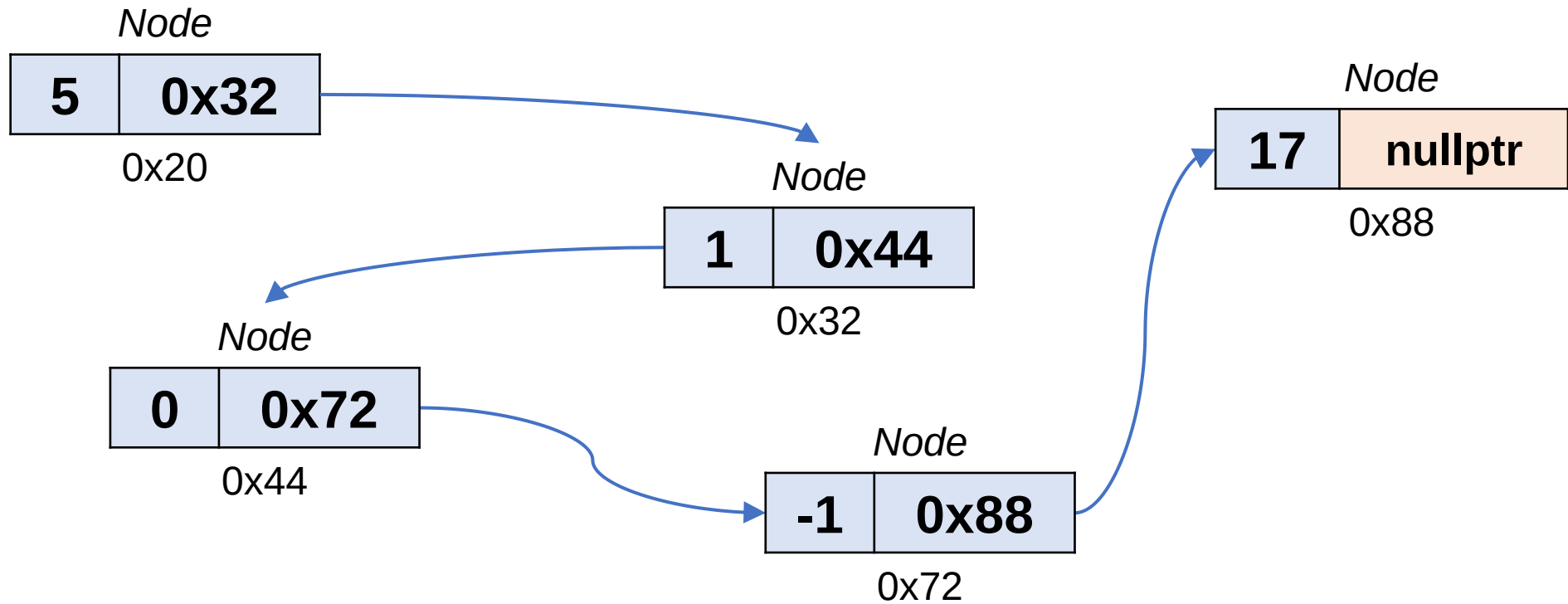
Question: What should be the value of **m_next** for the last Node?

```
class Node
{
    int m_data;
    Node* m_next;
};
```



While traversing, if you reach a **Node** whose **m_next** is **nullptr**, that means you have *reached the end* of the LL.

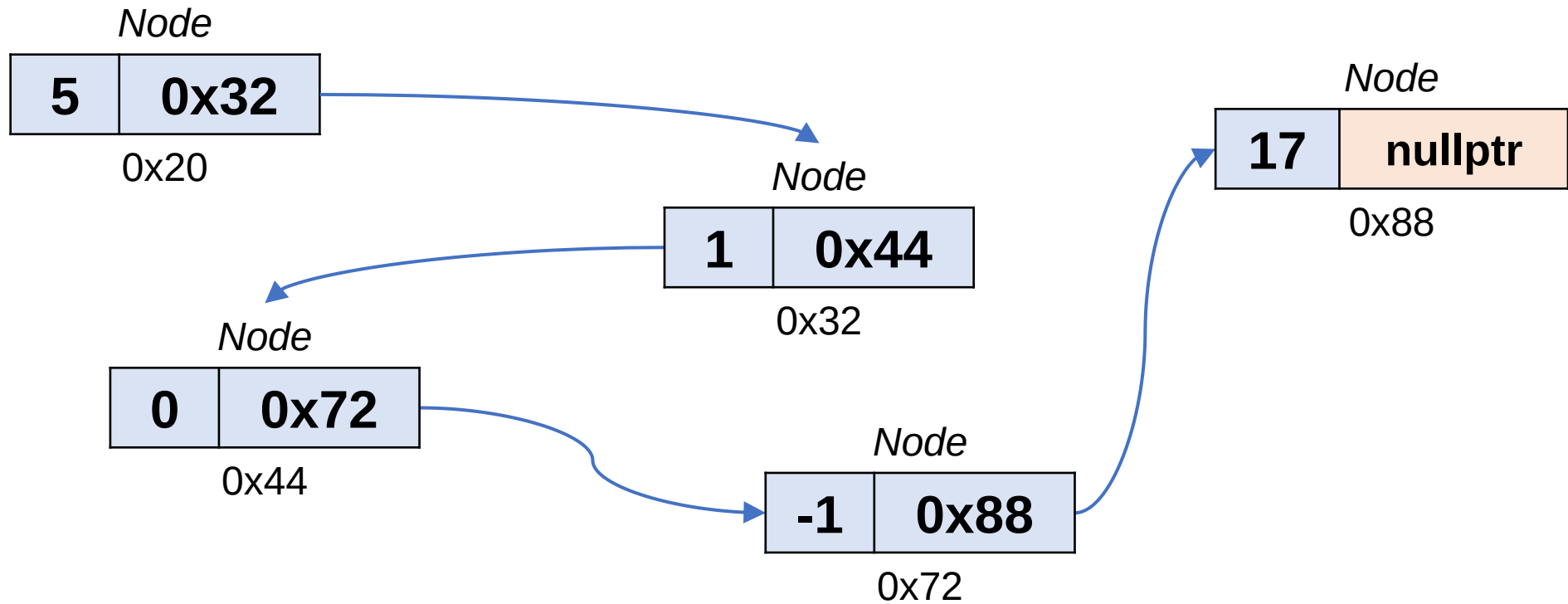
```
class Node
{
    int m_data;
    Node* m_next;
};
```



Note that I can only traverse a LL in **one direction**.
If I'm at a **Node**, I can only go to the "next" **Node**.

Question: In that case, where do I need to be for me to reach the start of my LL?

```
class Node
{
    int m_data;
    Node* m_next;
};
```

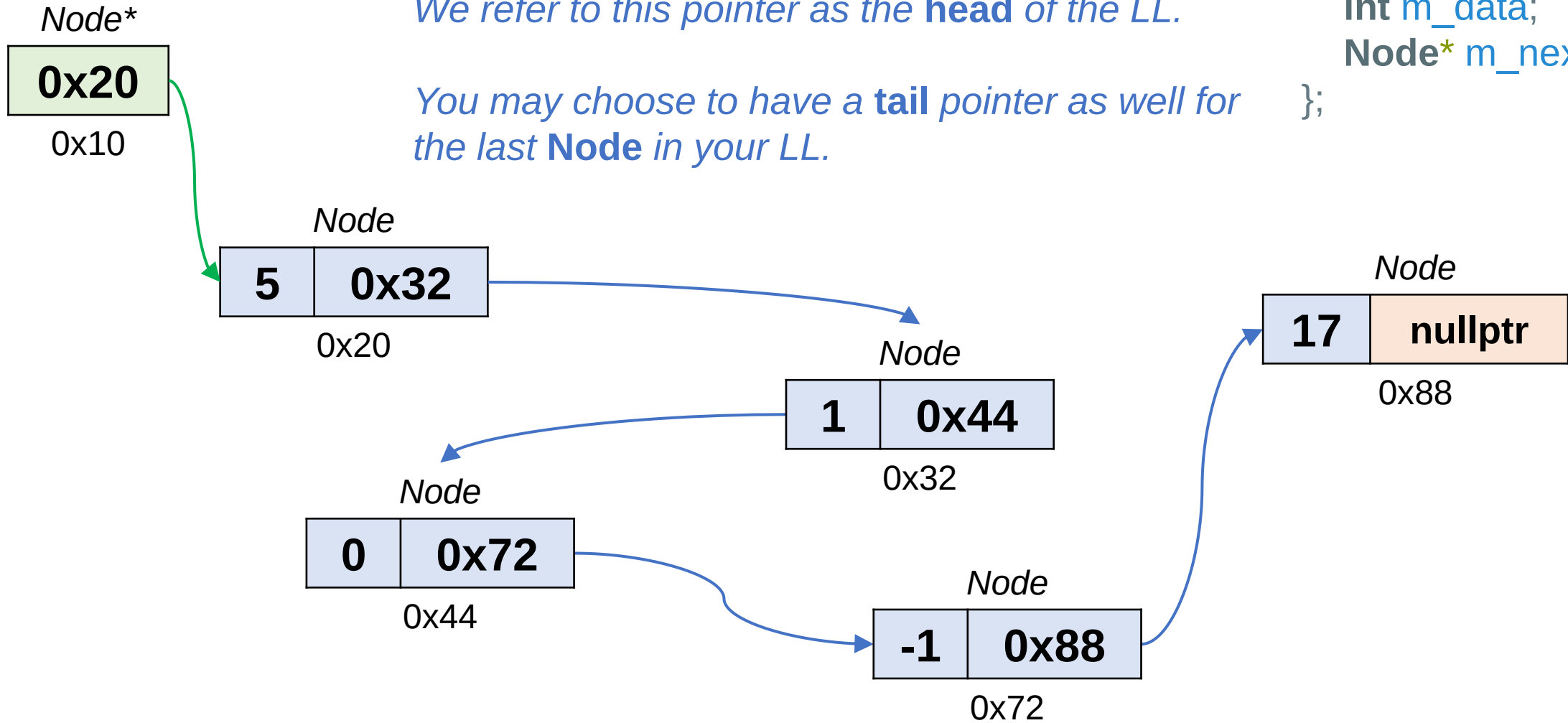


You require a **Node*** pointer that has the address of the first **Node** in your LL.

We refer to this pointer as the **head** of the LL.

You may choose to have a **tail** pointer as well for the last **Node** in your LL.

```
class Node
{
    int m_data;
    Node* m_next;
};
```

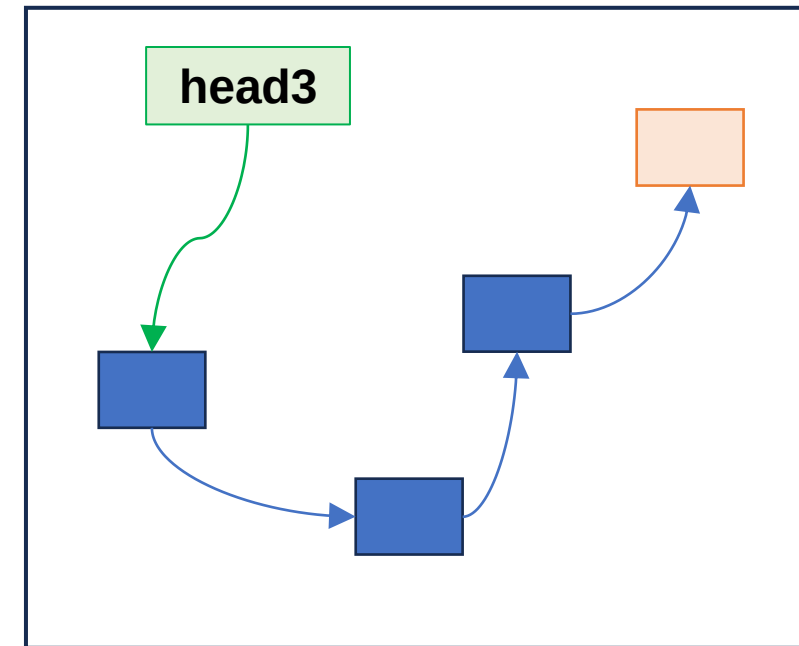
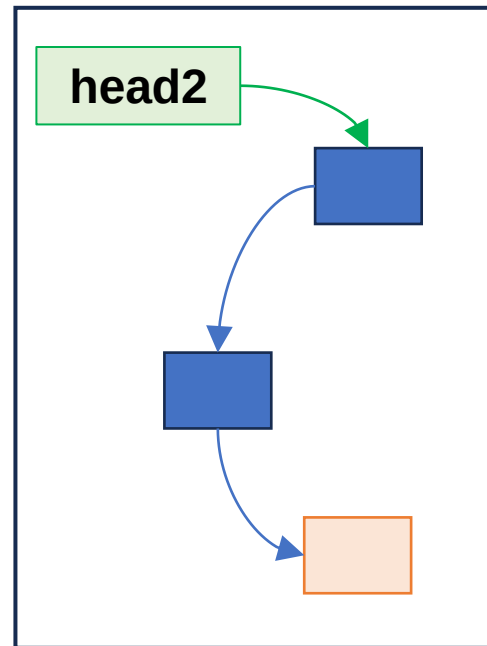
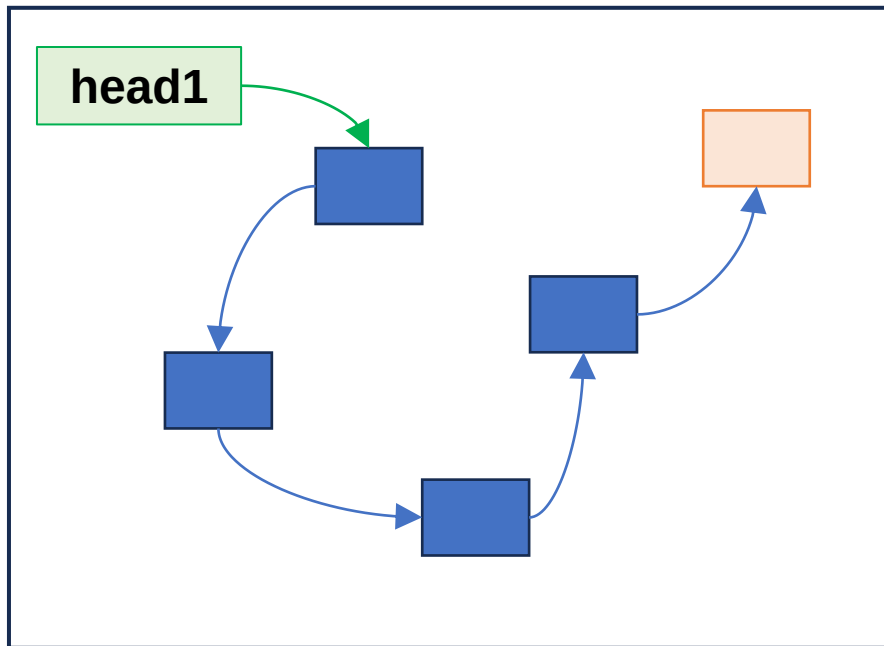


*Note that each of the following LLs are composed of **Nodes**.*

*However, each of these LL is a **separate entity** in its own right.*

*Based on this idea, we can actually consider each of these **LL as an object**, belonging to **class LL**.*

```
class Node
{
    int m_data;
    Node* m_next;
};
```



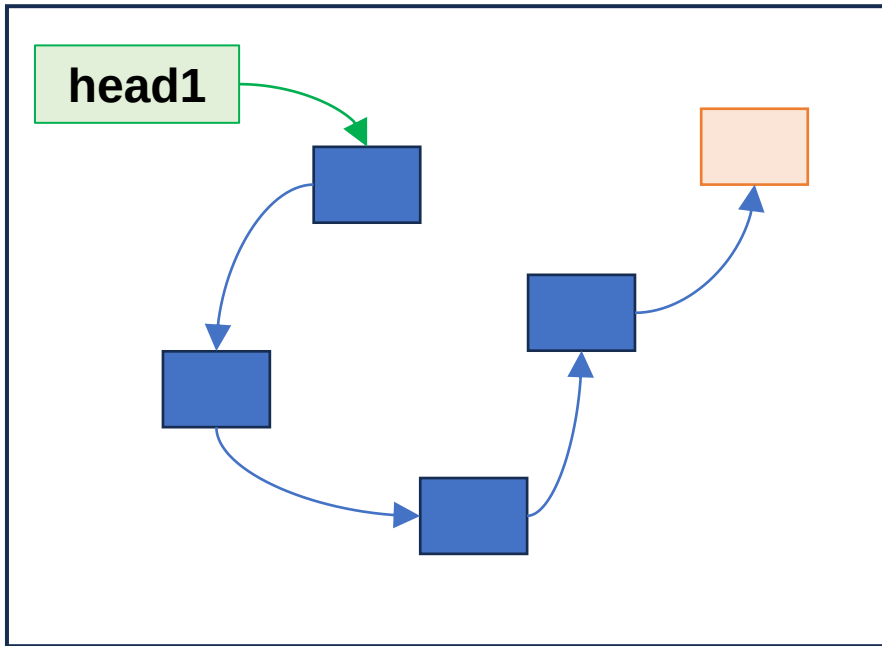
```
class LinkedList
{
    Node* m_head;

    // Methods
};
```

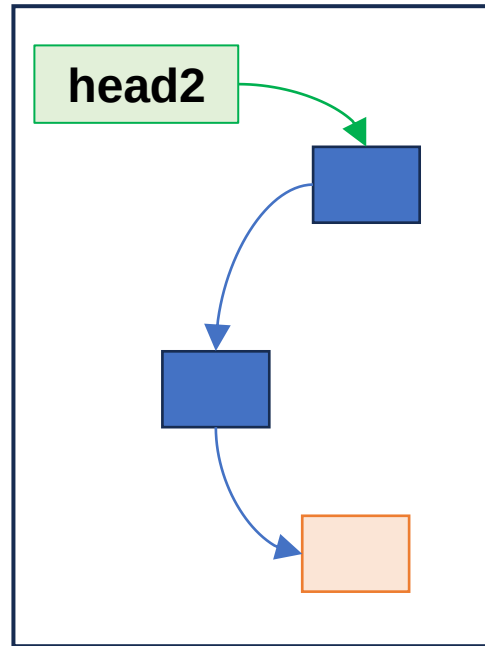
```
class Node
{
    int m_data;
    Node* m_next;

    // Methods
};
```

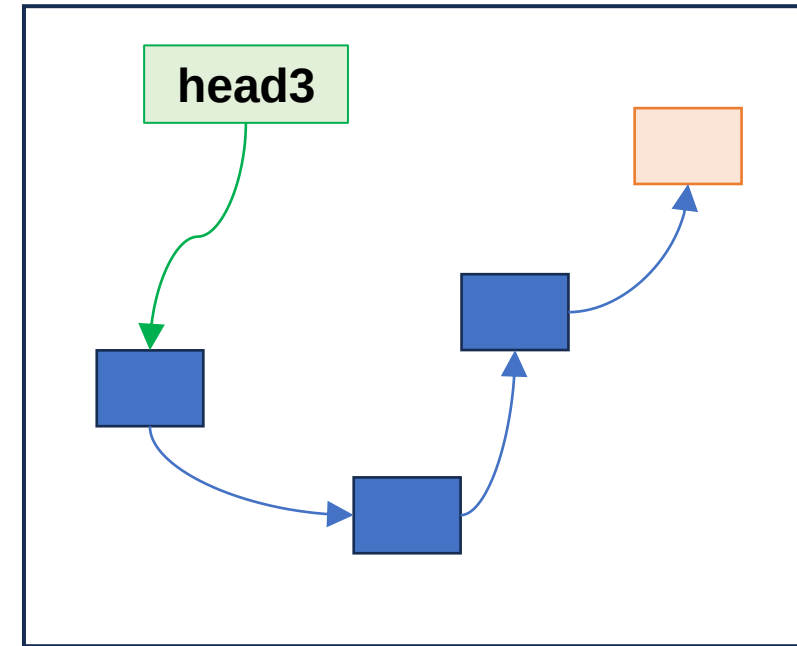
LinkedList list1;



LinkedList list2;



LinkedList list3;



Understanding Behaviors of a LL

*We will use the Queue ADT to understand this.
This is also what you have to do in this week's lab.*

Here is our **class Node**.

I have set the members here public, but they could easily be private as well.

If you are using private, you will need getters and setters for both the data members.

// For now, let the members be public.

// We will later see how to make Node "private" somehow.

class Node

{

public:

int m_data;

Node* m_next;

/ Because your members are public,
you can directly initialize them.*

*No need for a constructor. */*

};

Next is our **class LLQueue**.

Instead of a head pointer, I have a front (just renamed to reflect the Queue ADT more accurately). Additionally, I also have a rear pointer.

Note: *Regular linked lists may also have a tail pointer alongside the head pointer.*

```
class LLQueue
{
private:
    Node* m_front;    // dequeuing from here
    Node* m_rear;     // enqueueing from here

public:
    LLQueue() : m_front{ nullptr }, m_rear{ nullptr } {}

    // Class def is incomplete.
    // More functions coming...
};
```

*This is what we'll have when we create our **LLQueue** object*

*Note that both the **front** and **rear** pointers are part of the same object (**queue1** in this example).*

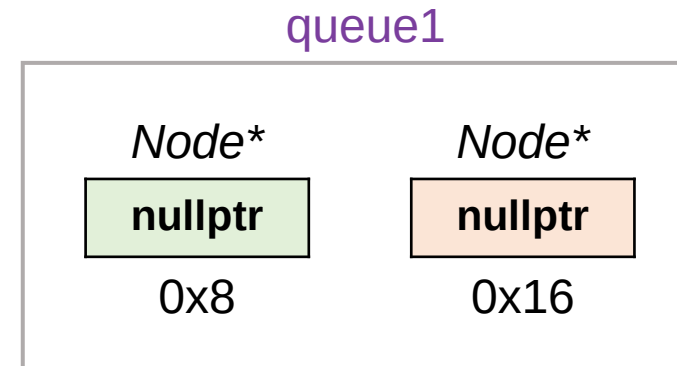
```
class LLQueue
{
private:
    Node* m_front;    // dequeuing from here
    Node* m_rear;     // enqueueing from here

public:
    LLQueue() : m_front{ nullptr }, m_rear{ nullptr } {}

    // Class def is incomplete.
    // More functions coming...
};
```

```
int main()
{
    LLQueue queue1;

    return 0;
}
```



Behaviors of the Queue ADT

- These are the behaviors that we need to implement.

void enqueue(**dtype** item);

dtype dequeue();

dtype front() **const**;

dtype rear() **const**;

bool is_empty() **const**;

Note

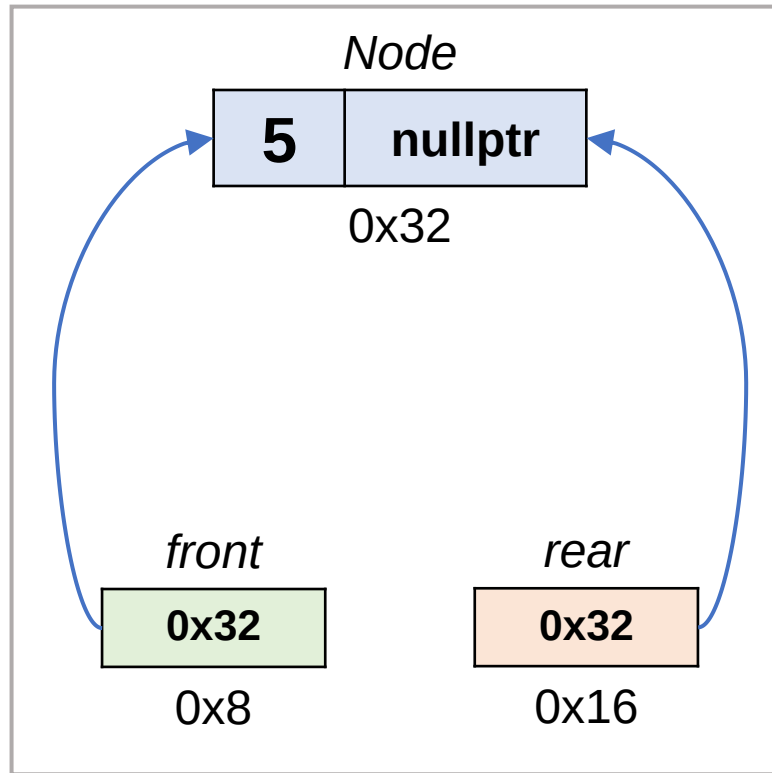
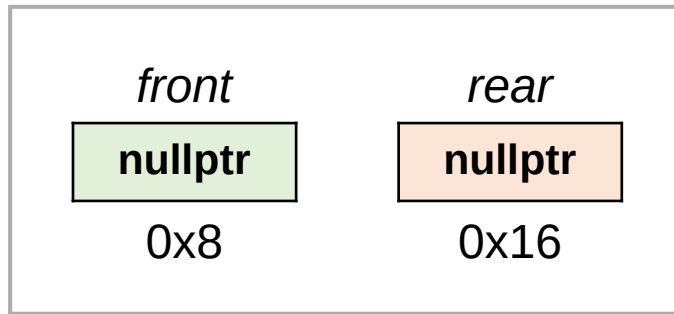
dtype refers to the data type of the elements that make up the linked list (or queue in this case).

- Could be others as well that could improve quality of life.

Behaviors of the Queue ADT

- In DSA, you saw the **Queue ADT** being implemented using the array data structure (*recall how an ADT was defined*).
- Now you will see that **same ADT** being implemented using the linked list data structure.
 - Motivation for this new data structure has already been shared at the start.
- Soon enough, we will implement the **List ADT** and see all types of functions being implemented using linked list as our choice of data structure.

Now let's see what should happen when we **queue1.enqueue(5)**
A Node should be instantiated with the user's input.
This Node will be the front and rear of the queue object.



```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);

    return 0;
}
```

*Now what should the function look like for this to happen
(I've removed all the other stuff from the class that we saw before).*

```
class LLQueue
{
public:
    void enqueue(int data)
    {
        // Create a Node object
        Node newNode{ data, nullptr };

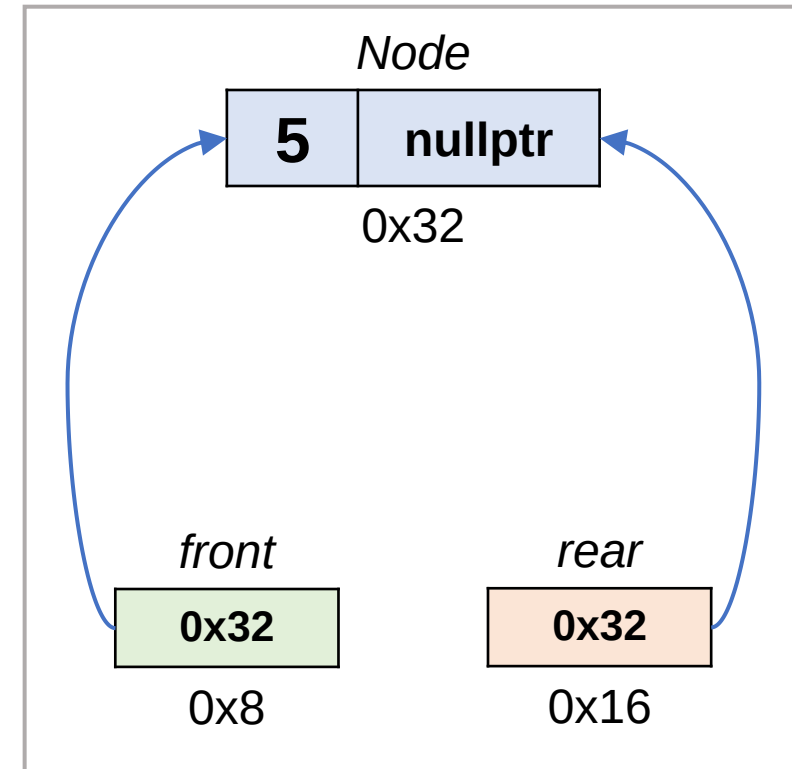
        // Two situations
        // 1- Empty linked list
        if(m_front == nullptr && m_rear == nullptr)
        {
            m_front = &newNode;
            m_rear = &newNode;
        }

        // 2- Not empty
    }
};
```

```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);

    return 0;
}
```



After this function finishes, I return to **main()**.

What will happen to the Node object that I created within **enqueue()**?

```
class LLQueue
{
public:
    void enqueue(int data)
    {
        // Create a Node object
        Node newNode{ data, nullptr };

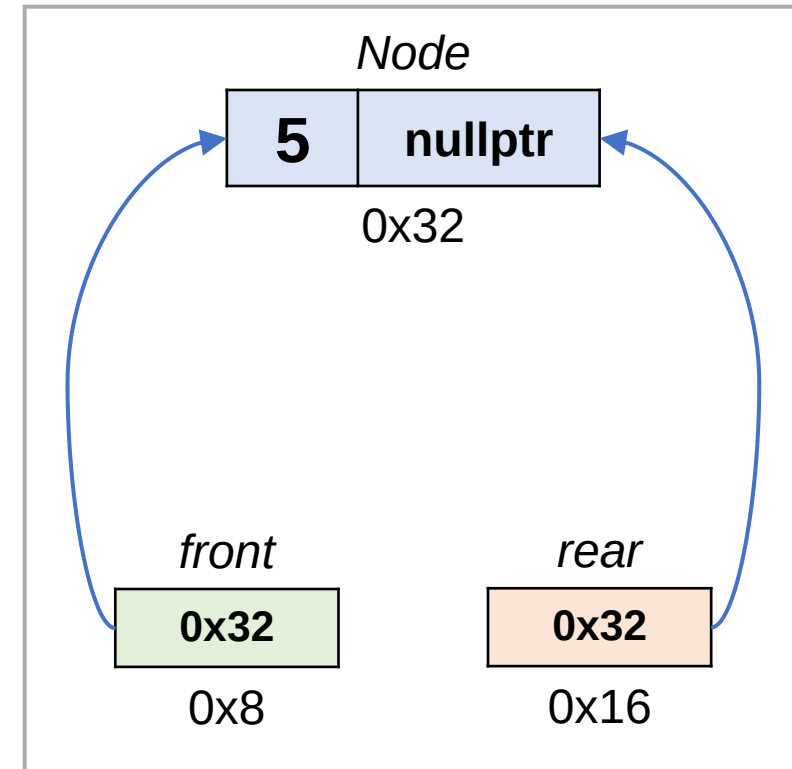
        // Two situations
        // 1- Empty linked list
        if(m_front == nullptr && m_rear == nullptr)
        {
            m_front = &newNode;
            m_rear = &newNode;
        }

        // 2- Not empty
    }
};
```

```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);

    return 0;
}
```



It's on the stack memory, so it automatically gets deallocated.
Dangling Pointers!

```
class LLQueue
{
public:
    void enqueue(int data)
    {
        // Create a Node object
        Node newNode{ data, nullptr };

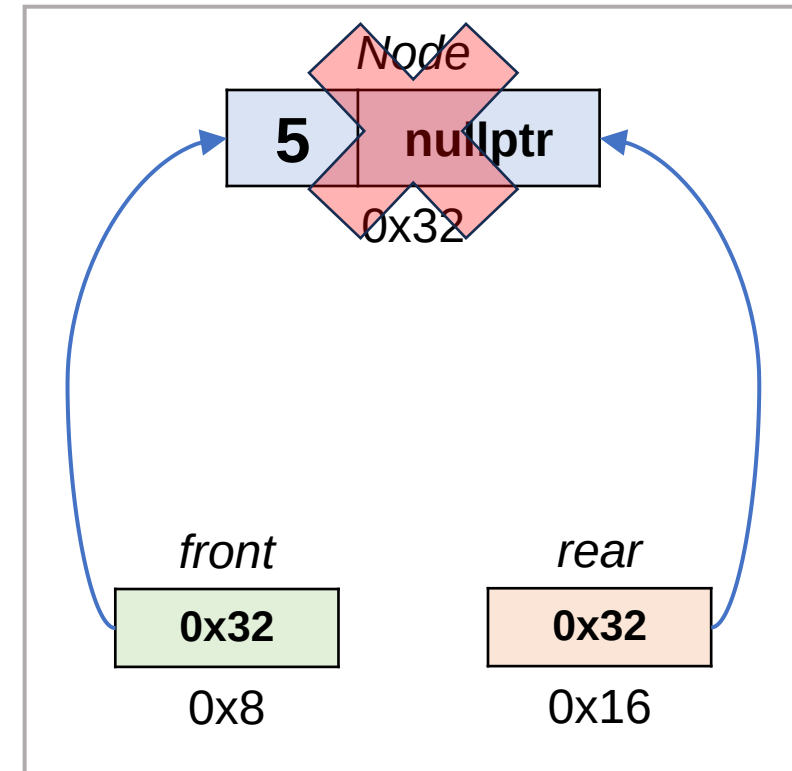
        // Two situations
        // 1- Empty linked list
        if(m_front == nullptr && m_rear == nullptr)
        {
            m_front = &newNode;
            m_rear = &newNode;
        }

        // 2- Not empty
    }
};
```

```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);

    return 0;
}
```



Create the Node on the heap memory (dynamically) so that it stays after the function ends.

This is an important idea. Also, you will need to **delete** all this memory too.

```
class LLQueue
{
public:
    void enqueue(int data)
    {
        // Create a Node object
        Node* newNode{ new Node }; // direct init also possible
        newNode->m_data = data;
        newNode->m_next = nullptr;

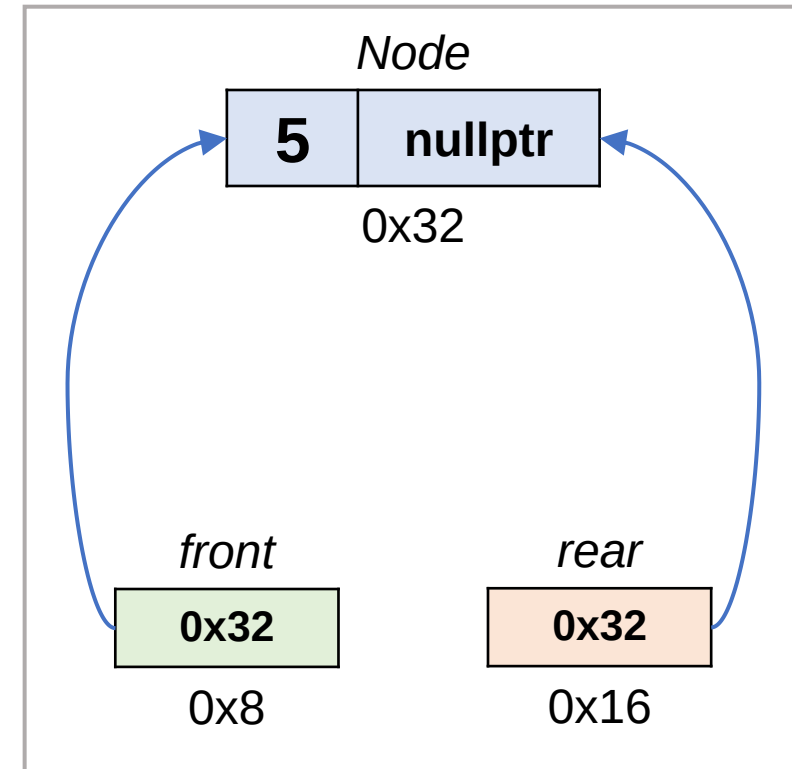
        // Two situations
        // 1- Empty linked list
        if(m_front == nullptr && m_rear == nullptr)
        {
            m_front = newNode;
            m_rear = newNode;
        }

        // 2- Not empty
    }
};
```

```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);

    return 0;
}
```



Also note how I'm accessing the data members through a pointer pointing at an object (the object itself could be on the stack or the heap).

`(*newNode).m_data == newNode->m_data`

```
class LLQueue
{
public:
    void enqueue(int data)
    {
        // Create a Node object
        Node* newNode{ new Node }; // direct init also possible
        newNode->m_data = data;
        newNode->m_next = nullptr;

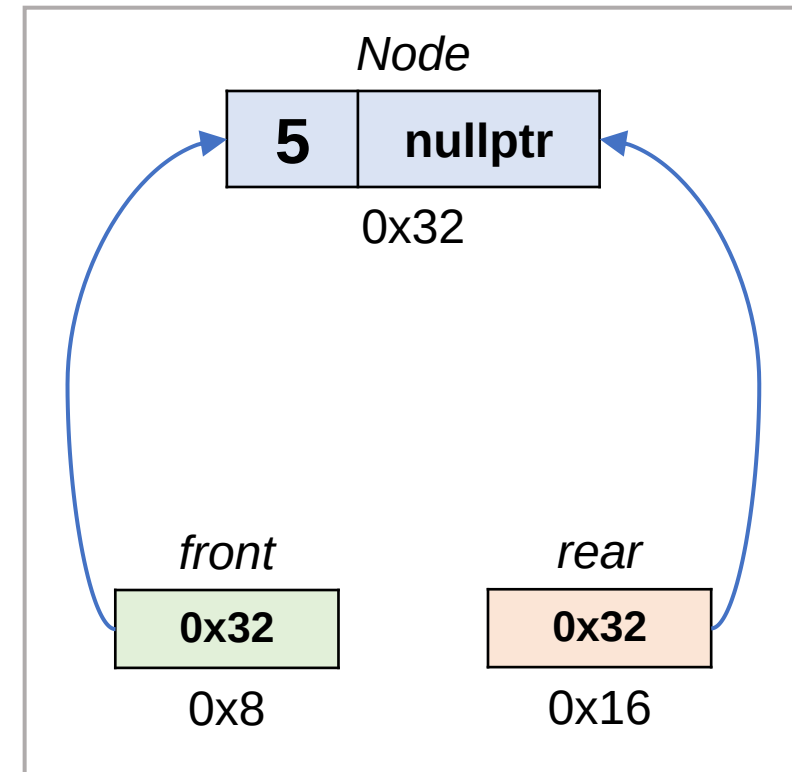
        // Two situations
        // 1- Empty linked list
        if(m_front == nullptr && m_rear == nullptr)
        {
            m_front = newNode;
            m_rear = newNode;
        }

        // 2- Not empty
    }
};
```

```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);

    return 0;
}
```

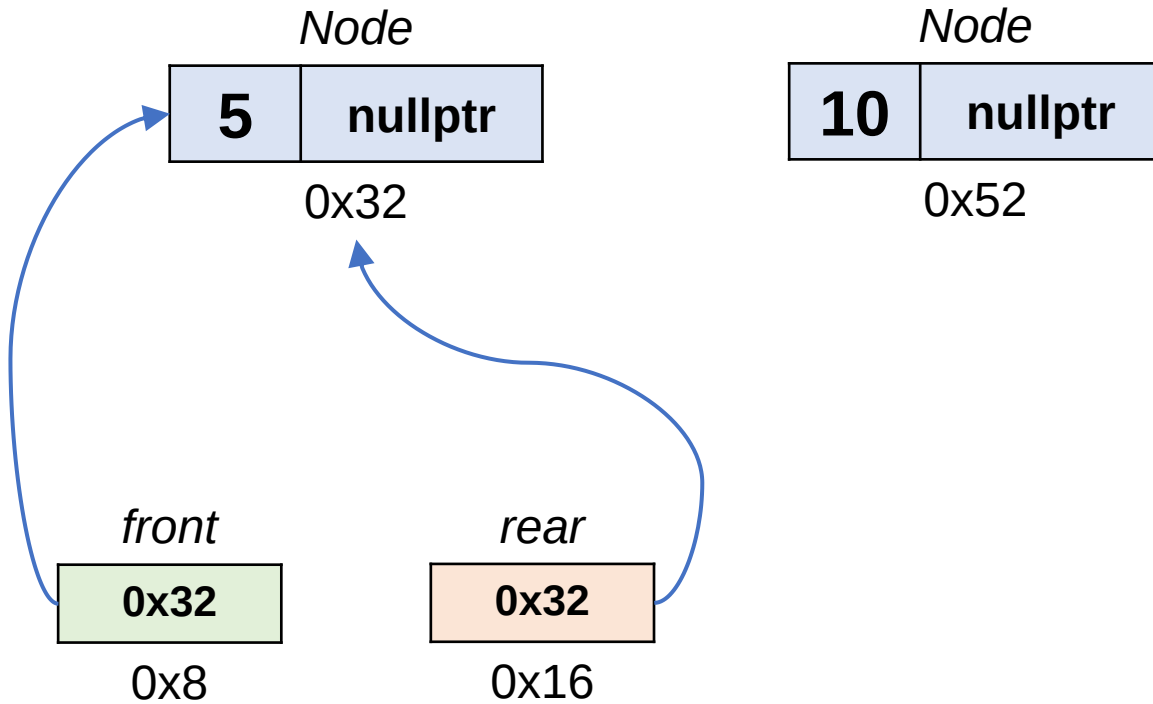


*Let's enqueue another object.
There is already a Node in our Queue. This will be handled differently.
Create a Node on the heap like before.*

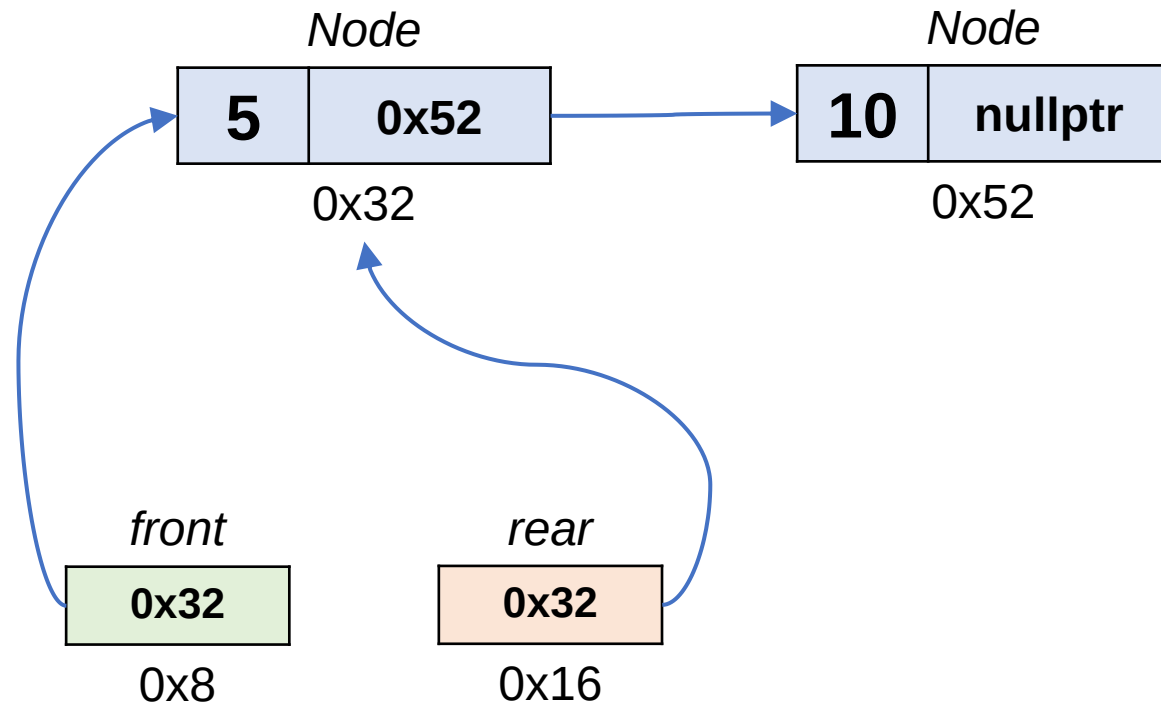
```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);

    return 0;
}
```



Update the **next** pointer of the very last Node (that is currently at the rear) in your Queue.



```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);

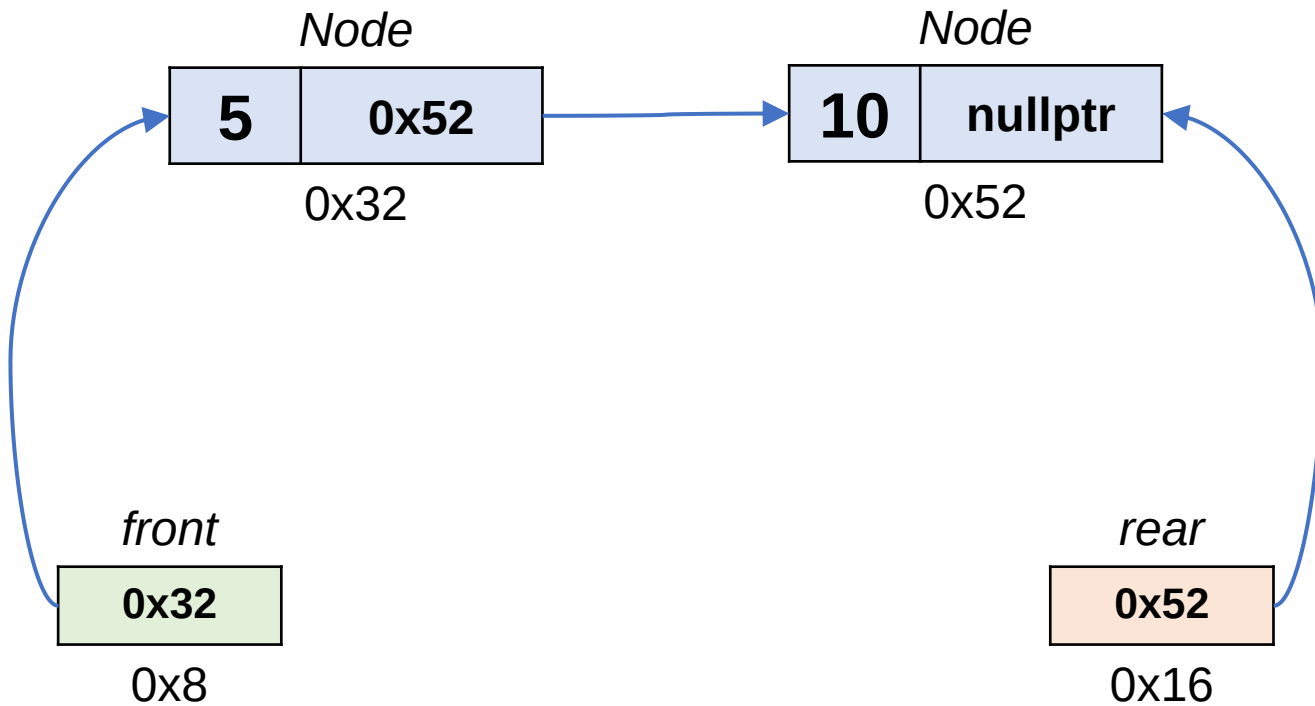
    return 0;
}
```

Finally, update the rear pointer so that it points to the newly added Node object in your Queue.
What if you updated the rear pointer first, and then the Node object's pointer?

```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);

    return 0;
}
```

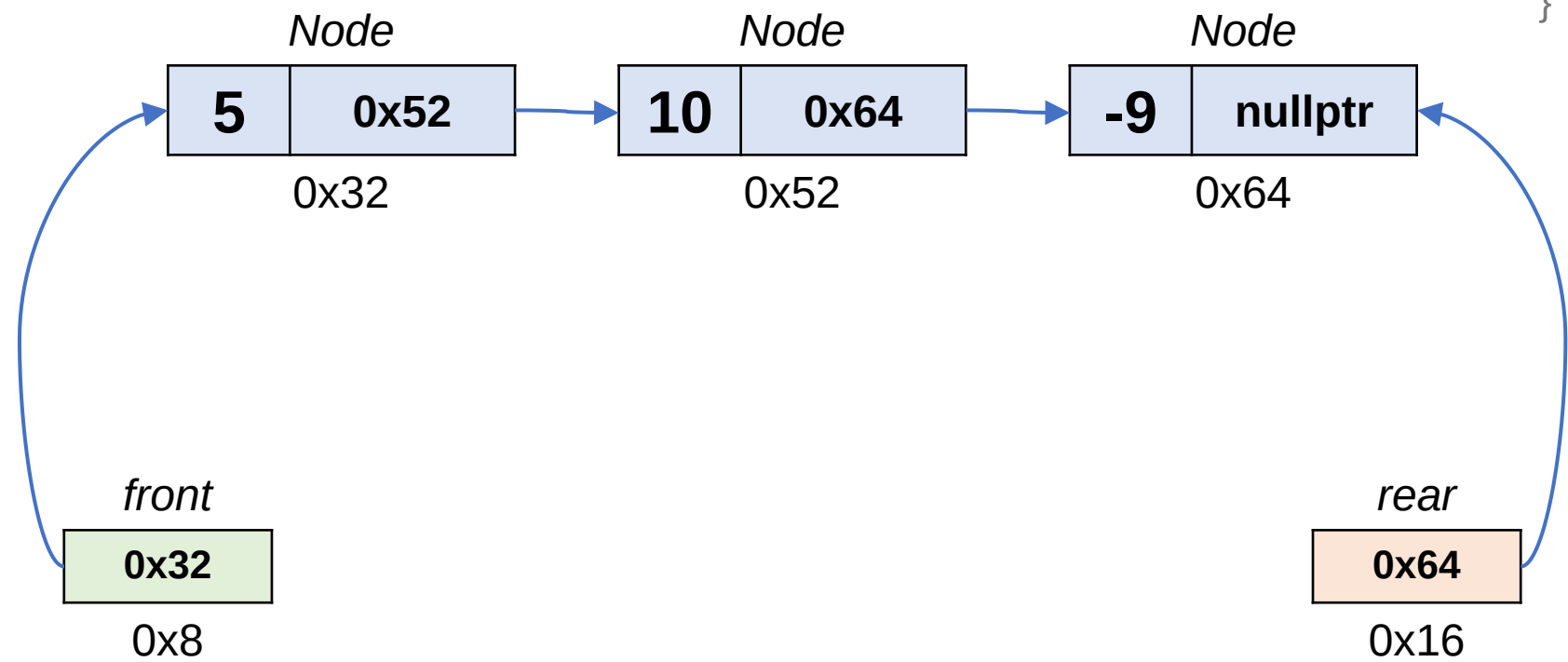


Let's enqueue one more Node in our Queue object.

```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    return 0;
}
```



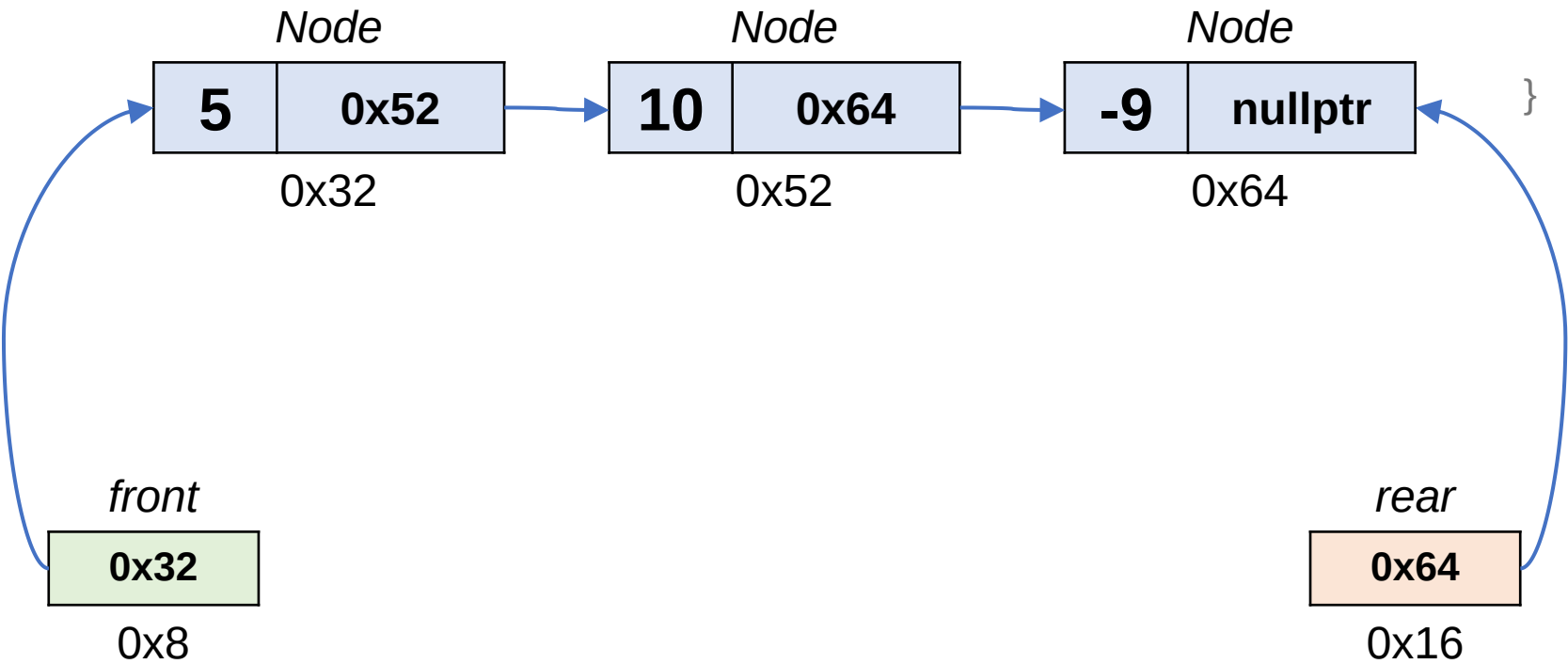
Let's start dequeuing now.
Dequeue happens from the front. So now, the rear will not change.

```
int main()
{
    LLQueue queue1;

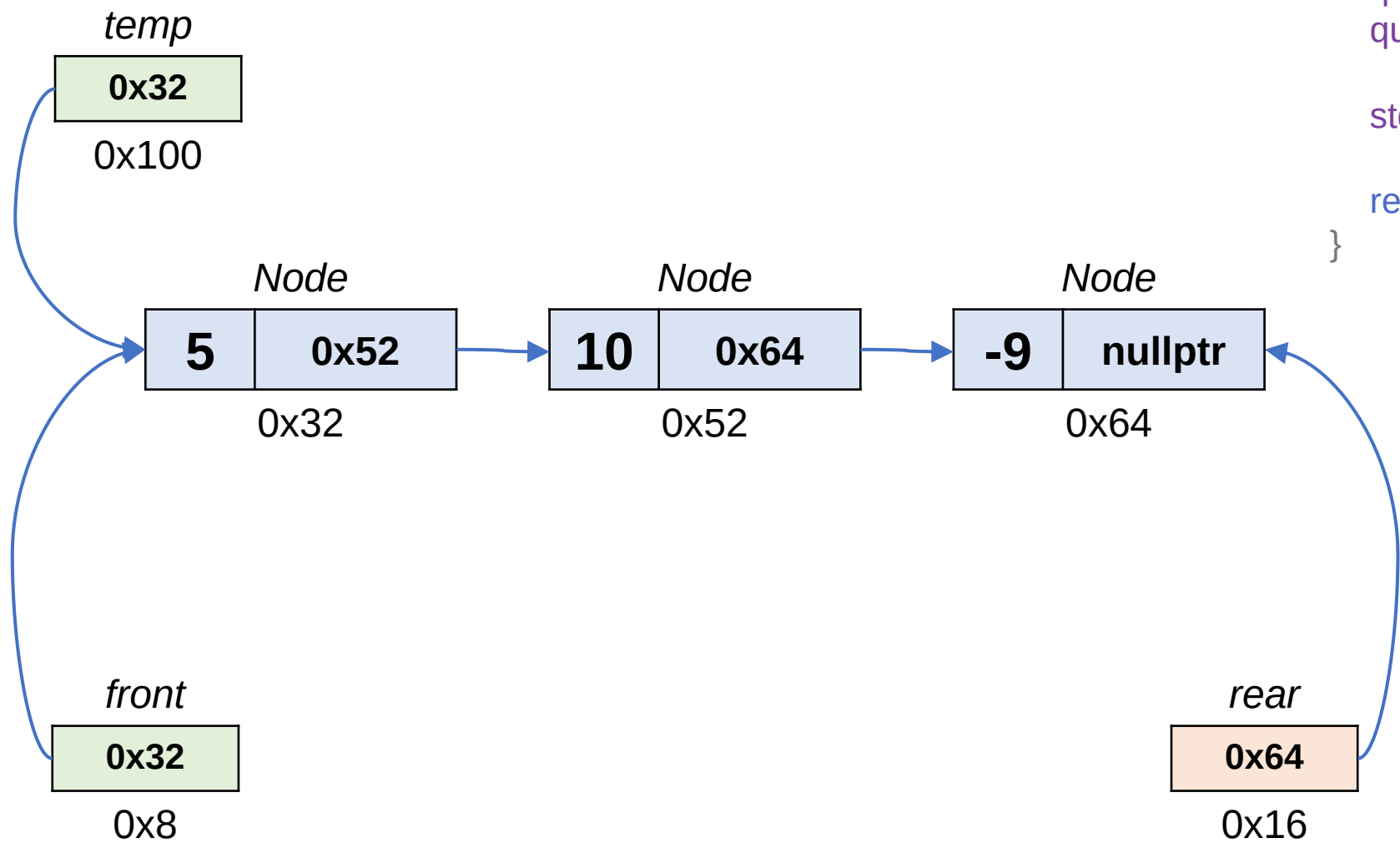
    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    std::cout << queue1.dequeue();

    return 0;
}
```



The way we do this is that we create a temporary Node-type pointer pointing at the front of the Queue.
Note that **temp** is a variable on the stack, local to **dequeue()**.



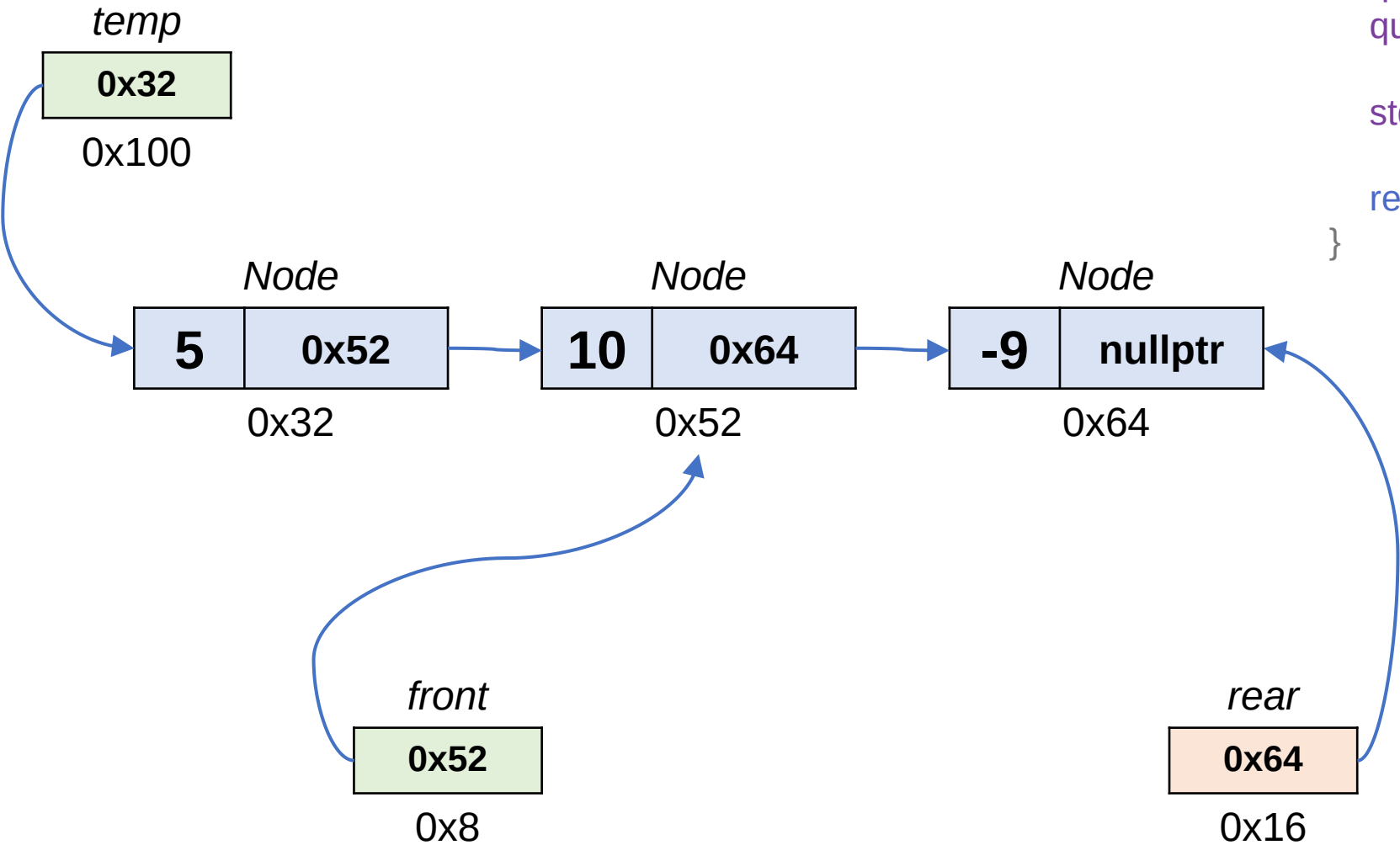
```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    std::cout << queue1.dequeue();

    return 0;
}
```

Update your **front** pointer like so.



```
int main()
{
    LLQueue queue1;

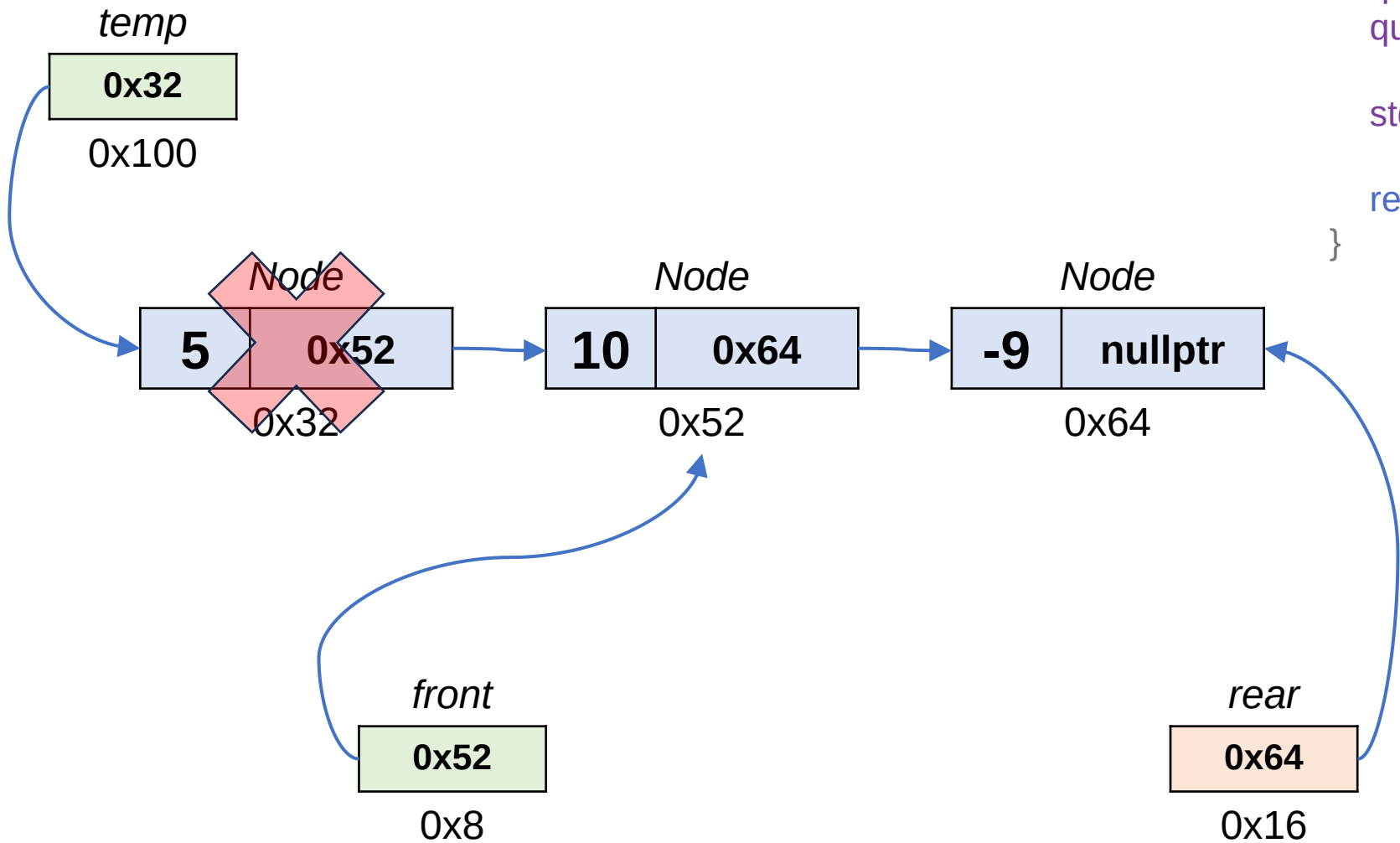
    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    std::cout << queue1.dequeue();

    return 0;
}
```

And finally, use **temp** to deallocate the Node object that it's pointing at.

delete temp;



```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

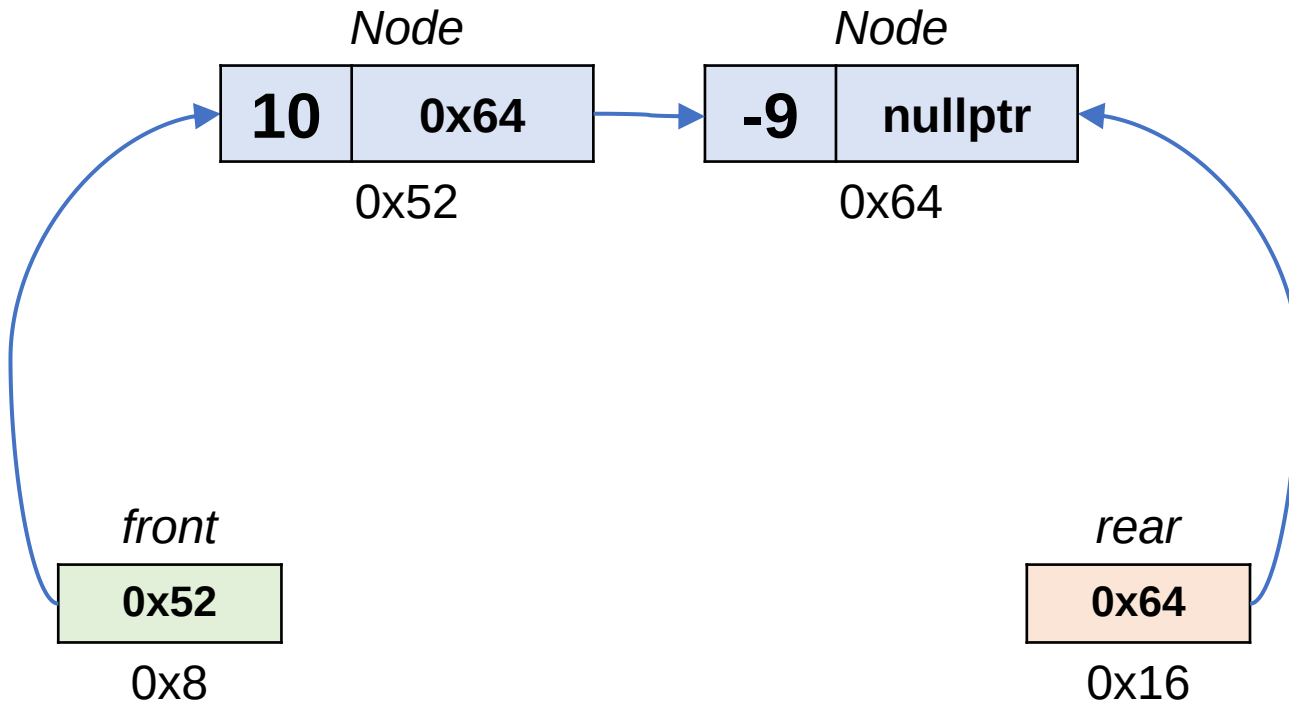
    std::cout << queue1.dequeue();

    return 0;
}
```

Note: There is an alternate logic to do this as well, which involves **temp** being initialized to be the second node in the linked list. Figure it out!

Deque operation would also return the data of the Node object that was just dequeued.

So make sure to save that data in a temporary variable and return that from the function.



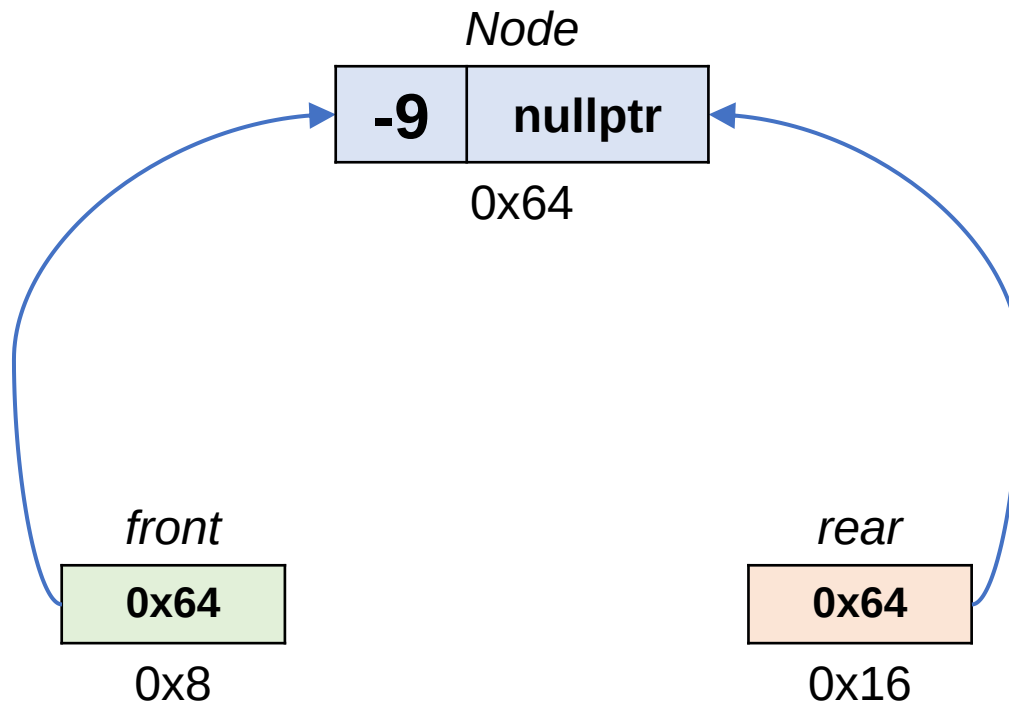
```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    std::cout << queue1.dequeue();

    return 0;
}
```

Another dequeue would do the following.



```
int main()
{
    LLQueue queue1;

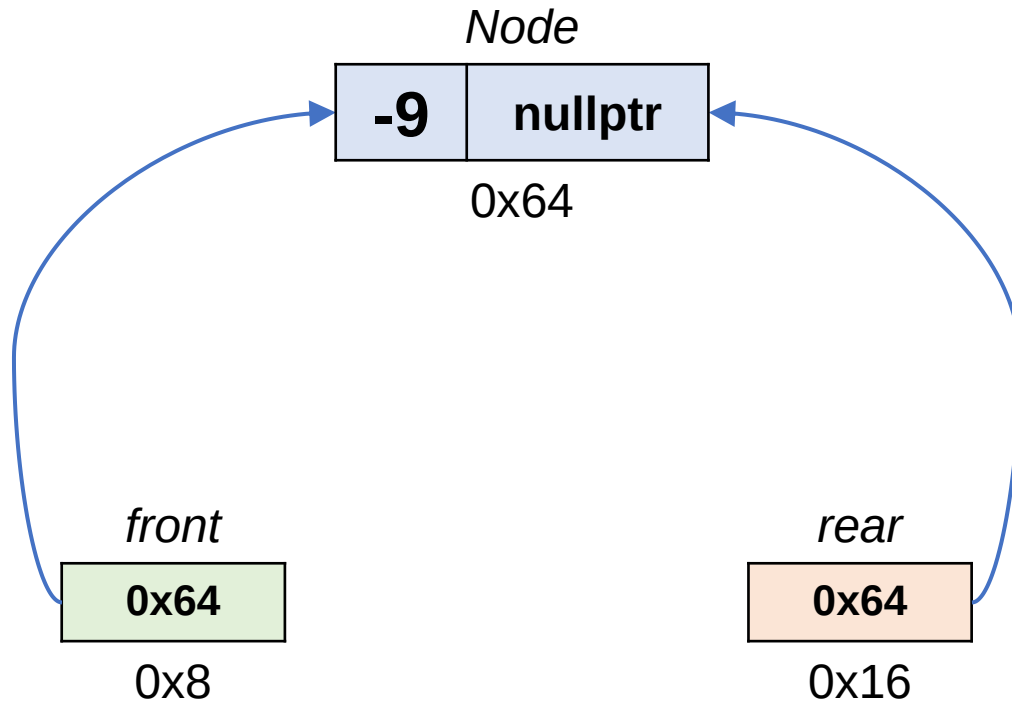
    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    std::cout << queue1.dequeue();
    std::cout << queue1.dequeue();

    return 0;
}
```

Previously, we saw how dequeue would work when there are **more than one Node objects** in your Queue.

Now, we will see how to dequeue when there is **only one Node** in your Queue.



```
int main()
{
    LLQueue queue1;

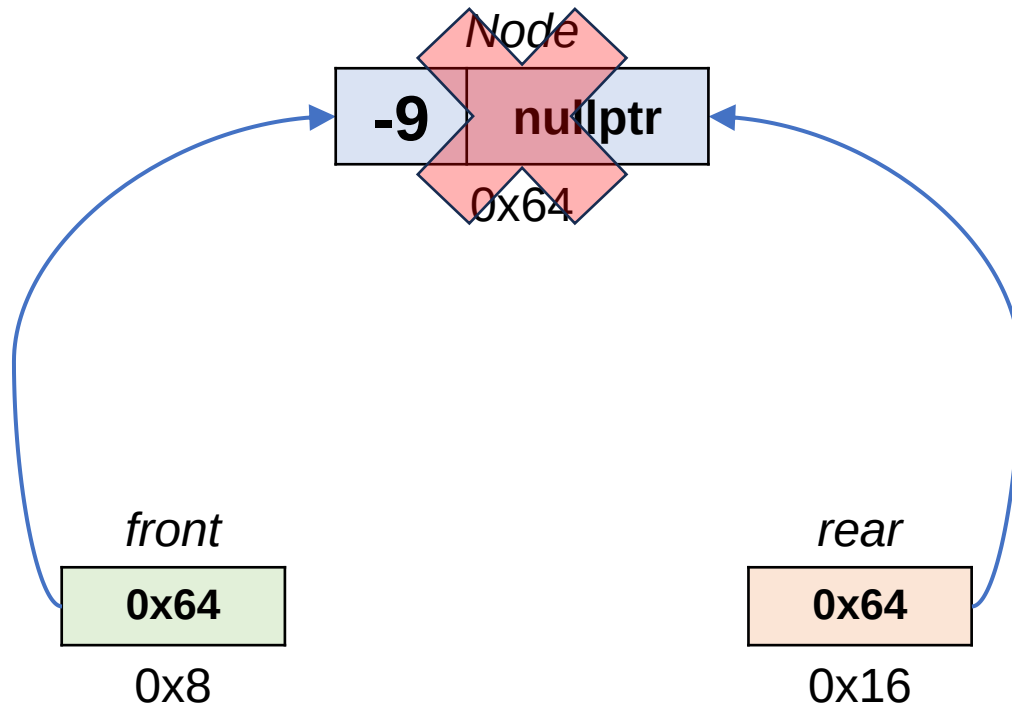
    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    std::cout << queue1.dequeue();
    std::cout << queue1.dequeue();
    std::cout << queue1.dequeue();

    return 0;
}
```

I could directly deallocate the Node object using the front pointer.

delete front;



```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    std::cout << queue1.dequeue();
    std::cout << queue1.dequeue();
    std::cout << queue1.dequeue();

    return 0;
}
```

Set both front and rear pointer to **nullptr**.



```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    std::cout << queue1.dequeue();
    std::cout << queue1.dequeue();
    std::cout << queue1.dequeue();

    return 0;
}
```

Note (self-read)

This could be one reason why you treat dequeue with a single Node as a separate case.

This is because you're modifying the rear pointer as well.

In the previous case with multiple Node objects, you did not touch the rear pointer.

*This is another case that you will account for while dequeuing – that is **dequeuing when your Queue is empty**.*

*You should give an error – “**queue underflow**”.
Therefore, you can just print a notification and return -1 from the function for example.*



```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    std::cout << queue1.dequeue();
    std::cout << queue1.dequeue();
    std::cout << queue1.dequeue();

    // Incorrect dequeue
    std::cout << queue1.dequeue();

    return 0;
}
```

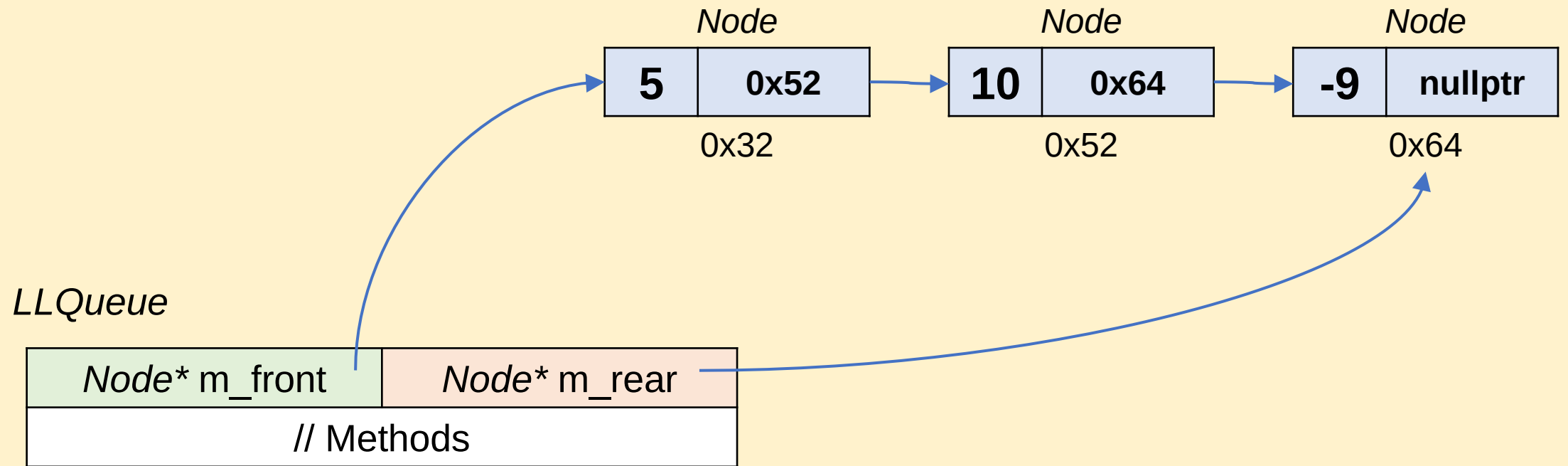
Notes for Self Study

- Please note that in all the visualizations shown earlier, the **front** and **rear** pointers are part of the **same object (queue1)** that belongs to **class LLQueue**.

queue1

<i>Node*</i> m_front	<i>Node*</i> m_rear
// Methods	

Notes for Self Study



Notes for Self Study

- Also note, that in this example the **queue1** object is created on the **stack memory**, so as it goes out of scope, the compiler will automatically deallocate it.
 - In order to deallocate it, the compiler will automatically call the dtor, **~LLQueue()**.
 - When the dtor finishes execution, the object will be destroyed.
- In your lab, the queue object was created **on the heap memory**.
 - So you must deallocate it using **delete**.
 - And when the compiler sees **delete**, it will then call **~LLQueue()**.
 - When the dtor finishes execution, the object will be destroyed.
- On observing your programs, some of you are doing “**delete[] temp;**” to deallocate your Nodes. **WRONG!**
 - These Nodes were allocated using **new**, not **new[]**. So they will be deallocated using **delete**, not **delete[]**.

Situations to account for while enqueueing and dequeuing

- Enqueueing has two situations.
 - *When the queue is empty.*
 - *When the queue is not empty.*
- Dequeueing has three situations.
 - *When the queue is empty.*
 - *When the queue has a single node.*
 - *When the queue has more than one node.*

Note

For a linked list to implement the List ADT (next class), we will see more conditions added on to these.

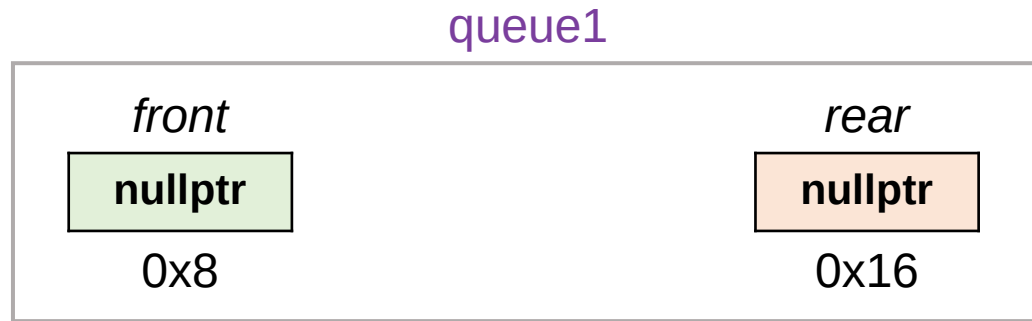
Freeing the memory

You should write a destructor to ensure this.

At the end of `main()`, the implicit destructor (provided by the compiler) gets called to destroy `queue1`.

Are there any memory problems that could happen?

No, because in `dequeue()` we freed all the heap memory.



```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    std::cout << queue1.dequeue();
    std::cout << queue1.dequeue();
    std::cout << queue1.dequeue();

    // Incorrect dequeue
    std::cout << queue1.dequeue();

    return 0;
}
```

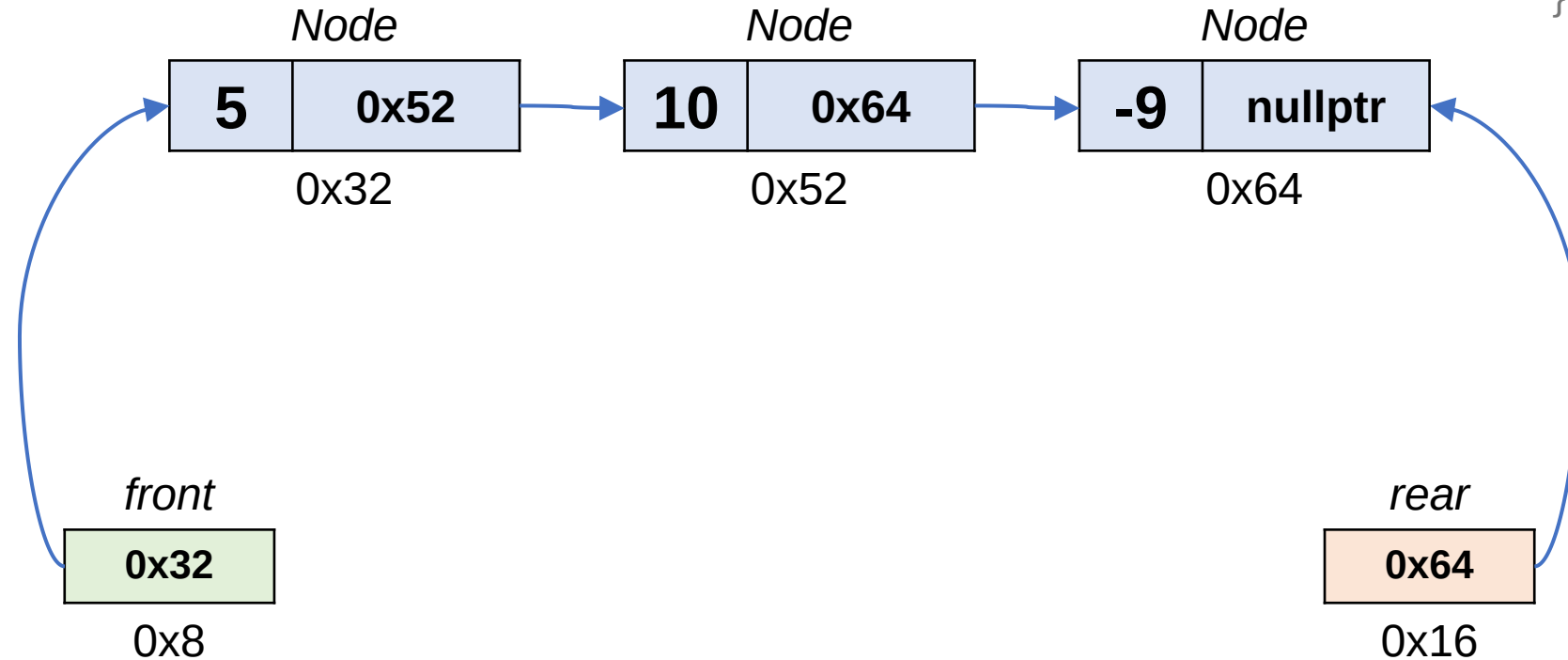
What if this was the current state of **queue1**, and it is about to go out of scope without all the dequeuing?

In general, you must **write your own destructor** to ensure that all of these Node objects on the heap get deallocated.

```
int main()
{
    LLQueue queue1;

    queue1.enqueue(5);
    queue1.enqueue(10);
    queue1.enqueue(-9);

    return 0;
}
```



Implementation of LLs in the Procedural Paradigm

You don't need classes to implement data structures!

No Classes Required!

- Instead of having a separate, **class LLQueue**, all the associated functions and data for the data structure can exist in the global scope (for data, it can be inside **main()**).
- For **class Node**, either wrap the attributes in a **struct** (available in C, which is not an OO language), or have them initialized as separate variables.